EEEE3001

Final Year Individual Project Dissertation

Intelligent Control for Laser Projection System Using Machine Learning

AUTHOR: Mr Ugonna Lemeh

SUPERVISOR: Dr. Sendy Phang

DATE: 07 May 2026

Executive Summary

The aim of this project was to develop an intelligent control system for laser projection, though a scattering medium, assessing the viability of two Machine Learning paradigms, a Supervised Learning Implementation, with a Multi-Layer Perceptron (**MLP**) and a reinforcement leaning model, a Proximal Policy Optimisation (**PPO**) agent to achieve a wavefront shaping solution to laser speckle through pattern reconstruction. An experimental optical setup comprising of a Texas Instruments DLP4710 Digital Micromirror Device (DMD), a scattering diffuser, a ZWO ASI120MM Mini CMOS camera and a 520 nm laser is used to acquire the dataset pairs, alongside an automated MATLAB-simulated approach to automation for hardware control and synchronisation with model training.

The **PPO** agent utilises PCA-compression techniques for observations and behavioural cloning warm-start, with benchmark values attributing the suitability of the model against an unseen 1000-sample test dataset to achieve poor performance. Results from analysis deem the behaviour of the **PPO** agent to exhibit a bimodal performance distribution, reconstruction errors, resulting in a 19% incidence of zero structural similarity to the target pattern, suggest the **PPO** model implemented is not sufficiently complex in tackling the complex nonlinearity of the speckle-DMD relationship at this hardware specification. Future work would involve deepening the model to account for the complexity of the Reinforcement Learning Architecture. The Supervised Learning Architectures involves two partitions of **MLP** models, the Forward and Inverse Controller model, similar models implementing different architectural choices, and which ultimately led to the assessment that both models were better suited for pattern reconstruction at this scale. The discrepancy between architectures realises in the final testing validation demonstrating that the Inverse controller design of **MLP** is the most effective for image-based wavefront shaping, also being the model with the shallowest architecture, with one total layer.

The results also highlight the suggestion that the **PPO** approach, that applied a sizeable amount of PCA compression necessary to maintain a computationally tractable learning process, fundamentally hindering the suitability reconstruction towards high-dimension patterns with up to 1024 configurable nodes. This project is believed to be the only implementation of MATLAB based deep learning architecture being used as a comparative solution to wavefront shaping and demonstrates a proof-of-concept approach through its accessibility limitations despite being scaled for variable-cost optical control without regarding phase-sensitive hardware.

Table of Contents

Executive Summary ii
Subject-Specific Terms iv
Tables v
Figures vii
1) Introduction 1
1.1 Project Background 1
1.2 Wavefront Shaping Development and State-of-Art 2
1.3 Aim and Specification 3
1.3.1 Deliverables 3
1.3.2 Methodology 4

1.3.3 Merit Figures 5
2) Design and Implementation 5
2.1 System Modelling 5
2.1.1 Experimental Setup 5
2.1.2 Experimental Logic Flow 11
2.2 System Development (Realisation and Sub-System Testing) 12
2.2.1 Machine Learning Model Architecture 12
2.2.2 Experimental Equipment Calibration 15
2.2.3 Dataset Development 18
2.2.4 Machine Learning Development 22
3) Final System Testing and Specification Validation 29
3.1 Testing Setup 29
3.2 Reinforcement Learning Performance Validation 30
3.2.1 Training Convergence Analysis 30
3.2.2 Aggregate Performance Metrics 31
3.2.3 Qualitative Validation 33
3.2.4 Summary and Specification Assumptions 35
3.3 MLP Performance Validation 35
3.3.1 Forward Model Validation 36
3.3.2 MLP Forward vs Inverse Controller Models Comparison 38
3.3.3 Summary and Specification Assumptions 41
3.4 Comparative Analysis 42
3.5 Error and Limitation Analysis 43
4) Conclusion 44
4.1 Reflection on the Result 44
4.1.1 Project State Review 44
4.2 Consideration of System Within a Larger Context 45
4.3 Reflection on Management 46
4.3.1 Risk Management 46
4.3.2 Skills Development 47
5) References 49
6) Appendix 51
6.1 Proforma 51
a) Progress Review 1 51
b) Progress Review 2 52
c) Progress Review 3 52
d) Progress Review 4 53

e) Progress Review 5 54
6.2 Code Snippets 54

Subject-Specific Terms

Laser Projection: The technology that involves illuminating a target scene using a coherent laser source. Laser projection is the primary method that enables modelled imaging.
Laser Speckle: Random granular interference patterns generated when a light source illuminates a diffusive surface, proliferating over larger distance of projection.
Speckle Contrast (Ks): The quantitative measure of the severity and prevalence of Laser Speckle, defined as the ratio of the standard deviation of intensity to the mean intensity.
Wavefront Shaping: Any optical technique that controls characteristics of light waves to manipulate light propagation to counteract speckle and is used for speckle reduction to correct distortion.
Spatial Light Modulator: A transmissive or reflective device, such as a Digital Micromirror Device (DMD), used to manipulate the amplitude, phase, or polarization of a light beam in a spatially varying manner.
Transmission Matrix (T): A mathematical operator describing how an incident light field is transformed into an output field after passing through a scattering medium, serving as the classical foundation for wavefront control.
Neural Networks: A Machine Learning architecture which is designed to mimic human brain processing by implementing a connection of layers made up of nodes (neurons) that abstract features and allow for pattern recognition.
Multi-Layer Perceptron (MLP): A feedforward artificial neural network consisting of an input layer, one or more hidden layers of neurons with nonlinear activation functions, and an output layer, trained via supervised learning to map speckle patterns to DMD configurations.
Hadamard Pattern: A binary orthogonal basis pattern used in computational imaging to systematically probe a medium, enabling efficient reconstruction of an image through mathematical inversion without redundancy.
Hidden Layer: An intermediate layer of neurons between the input and output layers that performs nonlinear transformations to extract complex features from the data.
Reinforcement Learning (RL): A branch of Machine Learning where an autonomous agent learns to make decisions by performing actions in an environment to maximize a cumulative reward.
Proximal Policy Optimization (PPO): An advanced reinforcement learning algorithm that improves training stability by using a clipped objective function to prevent the policy from changing too drastically in a single update.
Agent: The decision-making entity in reinforcement learning that interacts with the environment, observes states, and performs actions.
Environment: The external system or physical setup (such as the DMD and camera) which generates observations and reward signals.
Reward Function: A mathematical formula that provides a scalar value to the agent to indicate the success or failure of an action based on the quality of the resulting image.
Policy: The internal strategy or mapping used by the agent to determine the next action to take based on the current state of the environment.
Actor-Critic Architecture: A reinforcement learning structure comprising an Actor network that determines which action to take, and a Critic network that estimates the value of the current state, enabling stable policy improvement.
Principal Component Analysis (PCA): A dimensionality reduction technique that transforms high-dimensional data into a smaller set of uncorrelated components.
Behavioural Cloning (BC): A pre-training method where a reinforcement learning agent is first

trained via supervised learning on expert demonstration data, providing a warm-start policy that accelerates subsequent trial-and-error learning.
Epoch: One complete pass of the entire training dataset through the neural network, with multiple epochs required for the model to converge to a stable solution.
Overfitting: A modelling error that occurs when a network learns the training data and noise too perfectly, failing to accurately process new, unseen data.
Mean Squared Error (MSE): A loss function measuring the average squared difference between predicted and target pixel intensity values, serving as both the training objective and a primary evaluation metric.
Peak Signal-to-Noise Ratio (PSNR): An engineering metric quantifying the quality of image reconstruction, computed for normalised data to evaluate image reconstruction accuracy.
Structural Similarity Index (SSIM): A perception-based metric that evaluates image quality degradation by comparing image quality and structure between two images.
Parameter: Internal variables of a Machine Learning Model which the system adjusts and learns on its own from training data to find optimal values, made up of weights and biases which evaluate features that are abstracted from the input field.
Tractable: A calculation or algorithm is considered tractable if it can be solved practically within a reasonable amount of time, without an impossibly long processing period.
Ground Truth: The true target output image in which a Machine Learning Model is tasked to reconstruct, or is aimed to predict.
Backpropagation: Refers to the internal algorithm that allows neural networks to mathematically trace errors backwards to calculate how much each parameter has contributed to the mistake, in which tells the model how much to nudge each weight to get a better inference.

Tables

Table 1- Core Specification Deliverables 3
Table 2- Stretch Specification Deliverables 4
Table 3- System Specification Validation Thresholds and Performance Implications 5
Table 4- Component specifications and usage justification 6
Table 5- Performance comparison between phase-only SLMs and binary amplitude DMDs 8
Table 6- MATLAB vs Python regarding simulation 10
Table 7- Composition of Dataset and Class Distribution 18
Table 8- Normalisation workflow 21
Table 9- **MLP** Model layer technical specification 23
Table 10- **MLP** Training Hyperparameters configuration 23
Table 11- Specification differences between Forward model (former) and Inverse Model (latter) 24
Table 12- Forward **MLP** Model Architecture 26
Table 13- **PPO** model technical specification 27
Table 14- **PPO** Training Hyperparameters 27
Table 15- **PPO** Observation and Action Space Specification 28
Table 16- Sub-System Validation Objectives and Results 29
Table 17- Quantitative and qualitative diagnostic metrics for system validation 30
Table 18- **PPO** Aggregate test metrics 32
Table 19- Confusion matrix 33
Table 20- **PPO** Specification Validation 35
Table 21- **MLP** Forward Model Aggregate Test Metrics 37
Table 22- Confusion Matrix Comparison: **MLP** Forward vs Inverse Controller 39
Table 23- **MLP** Inverse Controller Aggregate Test Metrics 40

Table 24- **MLP** Forward Model Specification Validation 41
Table 25- **MLP** Inverse Controller Specification Validation 41
Table 26- Comparative metrics table where ↑ indicates a higher value is preferred and ↓ denotes a lower value is preferred 42
Table 27- Deliverable Completion Summary 44

Figures

Figure 1- Principle of wavefront shaping through scattering media 2
Figure 2- Experimental Setup diagram 7
Figure 3- Optical Bench for Physical Assembly of Setup 7
Figure 4- The read noise diagram for AR0130CS Mono Sensor with read noise(e-rms) and gain axes 9
Figure 5- supplementary optical baffling and custom-cut matte-black enclosure 9
Figure 6- Experimental Logic Flow Diagram 12
Figure 7- Forward model schematic 13
Figure 8- Inverse controller sub-model architecture 13
Figure 9- **PPO** model schematic 15
Figure 10- **(a)** Bullseye image generated in Microsoft PowerPoint displayed on DMD, **(b)** Corresponding output captured by ZWO camera displayed in ASICap 16
Figure 11- **(a)-(e)** Captured exposure calibration with E{-8.8, -7.3, -6.8, -2.3, 2.8} respectively 17
Figure 12- Speckle contrast overlay of an all-off speckle pattern 18
Figure 13- **(a)-(i)** Representative samples from each of the stratified pattern classes (left-to-right) corresponding to Table 7. 20
Figure 14- Representation of preprocessing pipeline through three MATLAB scripts 21
Figure 15- Episode reward trajectory via MATLAB Reinforcement Learning Training Monitor 31
Figure 16- SSIM per-sample distribution for **PPO** model 32
Figure 17- **(a)** PSNR Distribution **(b)** SSIM vs PSNR comparison 33
Figure 18- Example of best **(a)** and worst **(b)** SSIM values exhibited by the **PPO** model with corresponding speckle-ground truth pairs 34
Figure 19- Threshold Curve for **PPO** model 34
Figure 20- **PPO** Jaccard distribution histogram 35
Figure 21- Forward **MLP** Training and validation MSE loss over full training duration 36
Figure 22- **MLP** Forward and Inverse Controller Performance Distributions 37
Figure 23- **(a)** PSNR Distribution **(b)** Combined SSIM vs PSNR for **MLP** models 38
Figure 24- **MLP** Forward Model IoU Distribution 38
Figure 25- **MLP** Forward and Inverse Controller Threshold Sensitivity Curves 40
Figure 26- **MLP** Inverse Controller SSIM Distribution 41
Figure 27- Pixel-wise absolute error seen by Supervised Learning 43

1) Introduction

This section introduces the project background, evaluates the core engineering objectives, and reviews the structural specifications of the system. Additionally, it outlines the project deliverables and details the methodology proposed to address the challenges of wavefront control.

1.1 Project Background

This project aims to develop a system for intelligent control to optimise laser propagation and navigation through scattering media, while systematically evaluating the suitability of trained Machine Learning algorithms for wavefront shaping. Laser projection systems offer exceptional spatial coherence for high-fidelity imaging, serving as critical infrastructure in modern optical communications and precise control systems. However, coherent laser imaging systems inevitably encounter optical artifacts that degrade image quality, manifesting as granular interference patterns known as laser speckle.

Laser speckle arises from the mutual interference of scattered wavefronts when coherent light propagates through a stationary diffuser or an inhomogeneous scattering medium. This phenomenon stems from localised optical path differences that distort the relative phases of individual light rays passing through the transmissive object [1]. Within various engineering contexts, excessive speckle propagation scales rapidly, transitioning from an aesthetic nuisance to a severe operational constraint. This degradation is highly pronounced in biomedical imaging, LiDAR-based remote sensing, and precision laser projection. Because the presence of speckle introduces a quantifiable performance bottleneck, it decreases measurement accuracy, degrades effective spatial resolution, and severely limits overall system performance [2]. The mitigation of this unwanted granular interference remains a vital engineering objective [3]. In optical coherence tomography (OCT), speckle degrades fine structural details and limits the diagnostic sensitivity required to prevent false negatives during early-stage disease detection [4]. In LiDAR-based autonomous navigation, speckle propagation obscures critical target signatures, leading to safety-critical detection failures due to degraded signal precision [5]. In laser projection systems, these measurable obstructions establish a clear engineering demand for intelligent wavefront control to suppress speckle. This need has prompted agencies like the National Institute of Standards and Technology (NIST) to fund targeted research into active optical mitigation methods [6]. The global market for deep learning-based optical solutions is expanding rapidly to enable coherent wavefront imaging through scattering media. This development drives commercial demand for intelligent optical systems across autonomous navigation, free-space optical communications, and clinical endoscopy [7]. Neural networks possess structural architectures well-suited for speckle mitigation, leveraging feedforward mapping topologies capable of modelling complex, non-linear mappings. These networks learn to generate corrective phase or amplitude profiles through supervised training on paired datasets [8].

This project establishes an original comparative benchmark between two distinct Machine Learning paradigms implemented within a unified MATLAB environment: a supervised shallow Multi-Layer Perceptron (**MLP**) and a Proximal Policy Optimization (**PPO**) reinforcement learning agent as active controller. While Machine Learning models have been applied widely to various optical tasks, direct comparative benchmarks assessing MATLAB-integrated wavefront controllers for real-time speckle mitigation are presently highly limited. The developed system receives speckled, intensity-only images captured by a ZWO ASI120MM Mini camera, translates these inputs into corrective binary state commands broadcasted via a Digital Micromirror Device (DMD) display control interface, and processes them through the trained models to project a reconstructed, speckle-free target image at the sensor. The two paradigms are rigorously evaluated against quantitative metrics, focusing on Mean Squared Error (MSE), Structural Similarity Index (SSIM), and computational execution latency.

1.2 Wavefront Shaping Development and State-of-Art

Wavefront shaping is a foundational technique for imaging through highly scattering media. Historically, this has been achieved by explicitly characterizing the medium or iteratively optimizing the incident wavefront to counteract phase distortions, as pioneered by Vellekoop and Mosk [9]. By modelling the scattering medium as a linear, deterministic operator, the relationship between the incident optical field E_{in} and the scrambled output field E_{out} can be characterised through a transmission matrix **T** as shown in Equation (1)

1

$$E_{out} = \mathbf{T} \cdot E_{in}$$

where **T** represents the complex propagation operator of the medium [10]. By experimentally measuring **T,** precise focus or image reconstruction can be achieved by calculating the phase-conjugate wavefront of the incident field [11]. Consequently, a phase-modulated mask designed as the inverse transmission operator $\mathbf{T}^{-1}$ can be applied to the wavefront modulator to act as an active, intelligent controller.

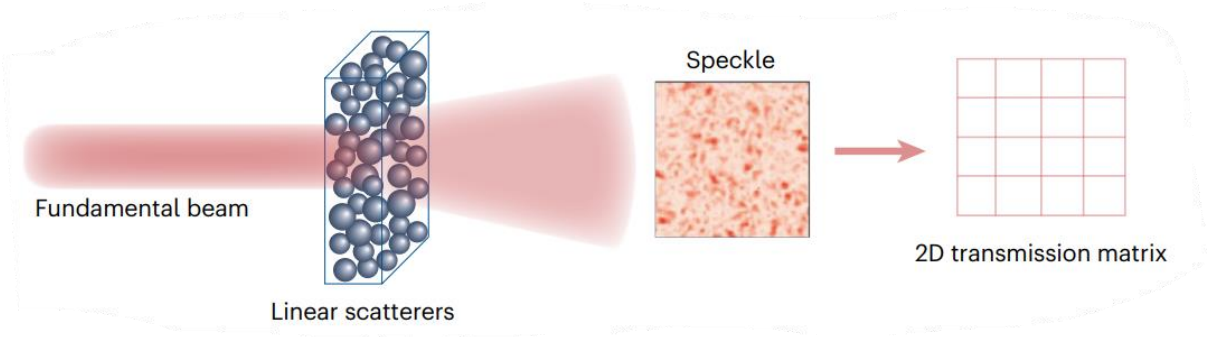


Figure 1- Principle of wavefront shaping through scattering media

Classical wavefront shaping techniques [9] that treat the scattering process as a phase-dependent inversion present severe practical difficulty [3]. This complexity arises because the corresponding transmission matrix is uniquely high dimensional, containing a vast array of controllable modes, requiring complex-valued phase calculations, and displaying extreme sensitivity to environmental perturbations. Contemporary approaches circumvent these iterative feedback loops and explicit phase calculations by leveraging deep learning. This paradigm decouples the computationally expensive training phase from the execution phase, enabling real-time inference. Neural networks can directly learn the underlying inverse mapping from observed speckle intensity patterns, generating corrective wavefront profiles in a single, high-speed forward pass [8]. The key advantage of these state-of-the-art Machine Learning models over classical algorithms [12] is their ability to bypass slow iterative searches. This transitions the computational bottleneck of wavefront control to an offline, highly intensive training phase, leaving the online operational inference phase computationally lightweight. Advanced convolutional neural networks (CNNs) often utilise skip-connection architectures to preserve fine optical textures, allowing them to map the complex, non-linear relationships between speckled camera inputs and optimal DMD corrective states to achieve robust control.

Despite these developments, deploying state-of-the-art deep learning architectures within physical, resource-constrained optical hardware remains limited. Many contemporary implementations assume unrestricted access to local hardware accelerators to run models characterised by inflated parameter counts, severe memory overheads, and massive training datasets. These requirements impose severe hardware bottlenecks. For edge applications requiring low-latency, portable wavefront control operated under modest desktop processors with strict CPU-only constraints, these large architectures are highly impractical.

The methodology adopted in this project addresses this limitation directly by demonstrating that intelligent wavefront control can be executed on low-cost, accessible hardware. By constraining model size and comparing shallow architectures against reinforcement learning agents, this work establishes a resource-efficient control benchmark. By designing explicitly for a system that utilises feedback from intensity-only

camera measurements and binary amplitude modulators, this work demonstrates that high-fidelity pattern reconstruction with limited resources is achievable.

1.3 Aim and Specification

The primary objective of this project is to develop and evaluate two distinct Machine Learning paradigms, a supervised Multi-Layer Perceptron (**MLP**) and an active Proximal Policy Optimization (**PPO**) reinforcement learning agent, to assess their capacity to achieve intelligent optical control by active wavefront modulation. The goal of this control system is to successfully mitigate speckle propagation through high-fidelity binary pattern reconstruction.

1.3.1 Deliverables

Core Deliverable	Requirements/Testing
D1. Replication and calibration of experimental hardware (Simulation/Experiment)	Calibrate the position of the laser and replicate the experimental architecture. This involves managing the TI DLP4710 DMD module via MATLAB and integrating the ZWO ASI120MM Mini camera natively over a stable USB connection. This integration establishes a baseline and generates a quantifiable speckle contrast ratio to be used in **D2**. Deliverable demonstrated in 2.2.2 Experimental Equipment Calibration.
D2. Automated image acquisition pipeline with managed preprocessing (Experiment)	Develop a MATLAB-based image capture system utilizing the active region of the ZWO Camera[1] to produce still images preprocessed to the correct input resolution for D3 and D4. This automated acquisition pipeline matches target Hadamard patterns to their corresponding output speckled images, building a structured dataset of 10,000 paired images (8,000 training / 1,000 validation / 1,000 unseen test). Deliverable demonstrated in 2.2.3 Dataset Development.
D3. Shallow **MLP** for speckle pattern processing (Application)	Train a Multi-Layer Perceptron[1] (**MLP**) architecture featuring a 64x64 downsampled input layer, two hidden dense layers with ReLU activation, and batch normalization to map speckled inputs to binary DMD patterns. The network must process complex speckle propagation features to intelligently map the scrambled input image to the clean output pattern, evaluated to a working standard on the unseen test set.
D4. PPO Reinforcement Learning agent for speckle control (Application)	Train an alternative, equivalent MATLAB Proximal Policy Optimization (**PPO**) agent using the Reinforcement Learning Toolbox[3]. To ensure parity with the **MLP**, the agent is designed to extract non-pictorial statistics and structures. To accommodate hardware limits, the agent runs in a simulated environment without physical actuation, utilizing direct reward feedback calculated against target ground-truth patterns.
D5. Comparative Analysis of ML approaches (Simulation)	Conduct a systematic comparison of the speckle mitigation performance of the **MLP** and **PPO** implementations. This evaluation examines convergence times, final speckle contrast metrics, and generalization behaviour when subjected to unseen speckle patterns. Statistical significance is assessed by comparing key specification metrics[4] to document operational trade-offs. Deliverable demonstrated in 3.4 Comparative Analysis.

Table 1- Core Specification Deliverables

[1]The ZWO ASI120MM Mini camera natively operates at 1280x960 pixels, but the physical constraints of the experimental setup (including lens-to-diffuser distance) yield a cropped active window of 640x640 pixels.

[2,3]The **MLP** [13] is implemented via the MATLAB Deep Learning Toolbox using the native trainnet function. The **PPO** agent is configured via the Reinforcement Learning Toolbox. While these toolboxes

establish our baseline benchmark, training larger, parameter-heavy architectures on external GPU accelerators would bypass MATLAB's built-in memory constraints at the cost of higher computational overhead.

[4]Quantitative evaluation and image quality metrics are calculated using the MATLAB Image Processing Toolbox to verify model outputs against the target ground truth patterns.

Stretch Deliverable	Requirements/Testing and Viability
D6. Enhance dataset with varied experimental geometries (Experiment)	Acquire an assortment of robust and geometrically diverse geometrical patterns for dataset inclusion to improve generalization capability.
D7. Higher-resolution scaling and model optimisation (Application)	Enhance dataset resolution up to a maximum of 1024 configurable modes in a 32x32 DMD pattern to increase difficulty from lower resolutions and evaluate scaling limits.
D8. Optimise to minimise computational load and maximise processing speeds (Application)	Reduce computational load and minimise processing speeds to their working minimal level, minimising operational latency.

Table 2- Stretch Specification Deliverables

1.3.2 Methodology

In this project, intelligent control of the laser projection system is achieved through utilisation and comparison of two Machine Learning approaches conducted and tested in MATLAB. For project management, weekly supervisor meeting would setup objectives that would necessitate a timeline, with a structural PowerPoint presentation being prepared each meeting documenting changes in project development, with Microsoft Teams being an avenue for file sharing and urgent troubleshooting needs. The planned methodology is split into three stages, reflecting the deliverable approaches that involve **Simulation, Experimentation** and **Application**.

a) Simulation

The simulation stage establishes idealised light propagation as it relates to the simulation environment and configuring prototypical architecture parameters and the control of the optical path DMD-camera relationship. This stage incorporates subsystem testing and calibration, enables further iteration of Model training and analysis of findings through application. Primarily involving the MATLAB computational infrastructure, this stage is first used to validate prior to hardware integration, and finalises the application assessments, allowing for a streamlined project process.

b) Experimentation

The experiment stage transitions into a focus on configurations that will assess how robust the project schema could ultimately prove under real-world physical constraints. During this phase, the optical setup and the hardware can be aligned to produce its optimal potential that will achieve the most suited construction of the dataset, and find configurations of hardware that will yield a clean and hardware-validated output response that creates the perfect conditions that will allow for the project's objective of intelligent control to be fulfilled.

c) Application

The application phase integrates the optimal configurations from the experimentation stage with the simulation models to allow for comparative and comprehensive analysis of the project goals. By assessing through Merit Figures and applied under computational constraints, the project's suitability against the core objectives can be objectively validated against comparative benchmarking, allowing for actionable insight for future optical control systems operating under similar conditions.

1.3.3 Merit Figures

Merit Figure	Specification Threshold	Implication
Binary Pixel Accuracy	≥ 70%	A binary accuracy below this threshold is marginally better than a naive classifier guessing all-OFF pixels, indicating a complete failure of the model's inference capacity.
Mean Squared Error (**MSE**)	≤ 0.08	A high MSE indicates a severe average pixel error, corresponding to a prediction that does not align structurally with the target binary pattern.
Peak Signal-to-Noise Ratio (**PSNR**)	Median PSNR ≥ 10 dB	A median value below 10 dB indicates that over half of the reconstructed test samples are of insufficient quality to overcome optical noise.
Structural Similarity Index (**SSIM**)	Mean SSIM ≥ 0.60	An SSIM below 0.60 indicates a loss of structural similarity during reconstruction, proving the controller cannot preserve spatial and structural features.
Jaccard Index	$IoU \geq 0.50$	An IoU below 0.50 demonstrates that less than half of the predicted and ground-truth active pixels overlap, indicating poor spatial feature resolution.

Table 3- System Specification Validation Thresholds and Performance Implications

Table 3 outlines the exact validation criteria a model must satisfy to demonstrate a successful control capability on unseen datasets. A failure to meet these thresholds highlights fundamental weaknesses in the model's architecture. However, evaluating performance via a multi-metric framework is critical because partial successes still provide valuable diagnostic insights. For example, a model might achieve high binary accuracy by correctly predicting background pixels while scoring poorly on SSIM due to a failure to preserve structural boundaries. Utilising multiple metrics ensures that classification and reconstruction quality metrics are assessed holistically, avoiding the analytical blind spots of single-metric evaluation.

2) Design and Implementation

2.1 System Modelling

2.1.1 Experimental Setup

The optical bench, illustrated schematically in Figure 2 and photographed in Figure 3, was initially developed by Mr Gabriel Pacheco [14] and has been adapted to fully accommodate controlled scattering and high-fidelity intensity capture. Unlike classical experimental setups that utilise devices designed to control complex phase information and light polarisation, a Digital Micromirror Device (DMD) provides light modulation.

Component	Model/Specification	Justification
Laser Source	VLM-520-03 LPT (520 nm, CW)	Provides coherent illumination as a green laser beam as an input to generate laser projection without spatial distortion.
DMD	Texas Instruments DLP4710	Chosen with knowledge of binary switching speed and MATLAB integration through HDMI compatibility under software control.
Neutral Density Filter	-	Attenuates incident power of the laser to limit CMOS sensor saturation and operating within viable SNR.
Beam Expander Lens	-	Expands laser beam to uniformly illuminate DMD display area for macropixel activation.
Focusing Light Lens	-	Concentrated scattered beam onto camera sensor plane, allowing for a focused capture.
Scattering Diffuser	-	Positioned after focusing lens to generate stationary speckle across the projected image, matching Goodman's fully developed regimen [15].
Light-Obstructive Tunnel	Matte black card, custom cut	Forms tight enclosure from Focusing Light Lens to camera aperture to suppress unwanted incident and background light that would otherwise introduce additive offsets and unwanted variables in the camera capture.
CMOS Camera	ZWO ASI120MM Mini	Framerate and USB integration via MATLAB selected for sub-second high-speed image capture and is used with 800x640 px capture.
Computer	Portable laptop with HDMI configuration and USB 3.0 functionality	CPU: Intel® Core™ i5-1135G7 @ 2.40 GHz RAM:32 GB GPU: None (dedicated 128 MB graphics) Operating System: Win 11 MATLAB Version: R2025b

Table 4- Component specifications and usage justification

This instrumentation choice fundamentally constrains the system to intensity-only measurements. However, as established in a) Digital Micromirror Device, the Machine Learning approaches adopted in this project learn the inverse scattering mapping without requiring explicit phase information. The components listed in Table 4 together induce deterministic speckle propagation, with each element serving a specific role in conditioning the optical path.

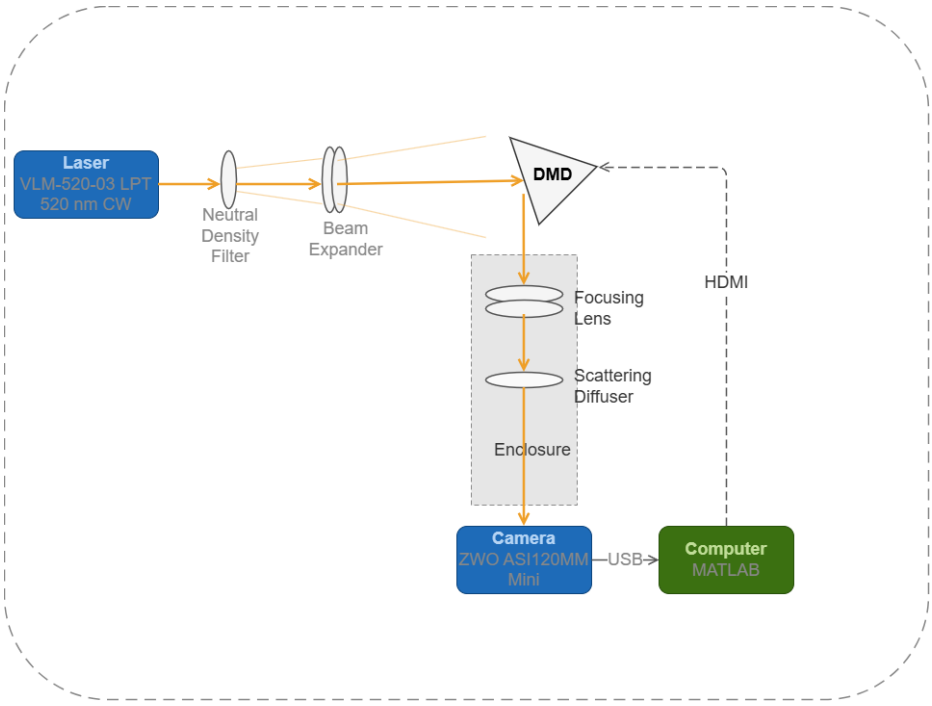


Figure 2- Experimental Setup diagram

The method of coherent light propagation throughout the experimental apparatus comprises a laser source and imaging optics to project light to a Digital Micromirror Device, connected via USB serial communication protocols to a laptop system and MATLAB computational control software. Figure 2 details an abstract flow of process from hardware to software integration, surmising the laser activation with the beam being spatially expanded from a single point to an expanded projection, in order to illuminate the entire active aperture of the 1920x1080 resolution DMD surface, which is reflected at an angle through the enclosure module to the ZWO camera aperture, completing the closed-loop laser propagation process. Interaction with the physical system is conducted through the camera's USB 3.0 connection with the computer using MATLAB as an independent control node. This node simultaneously coordinates pattern projections on the DMD through an HDMI connection, allowing for automated execution to fully synchronise the data acquisition to model evaluation pipeline, chronicled in the logic flow in 2.1.2 Experimental Logic Flow.

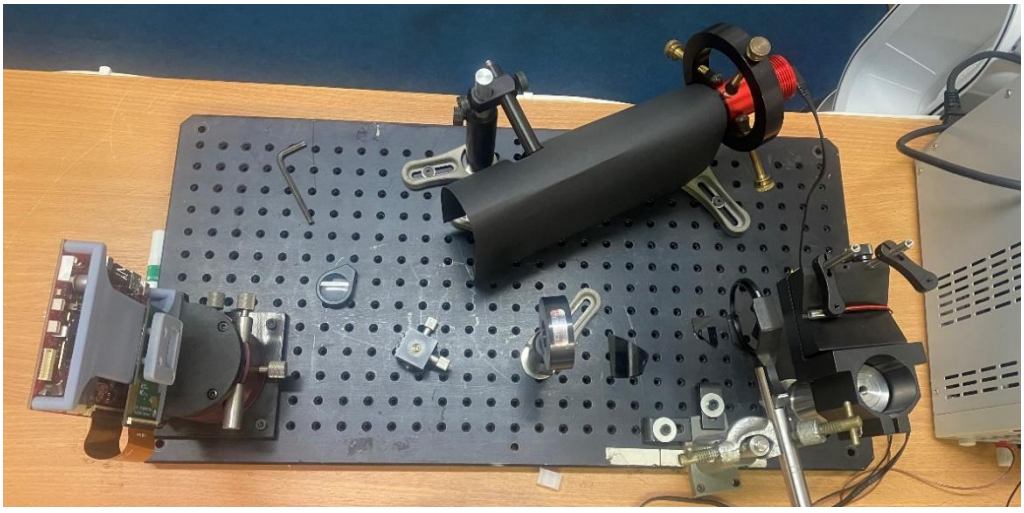


Figure 3- Optical Bench for Physical Assembly of Setup

a) Digital Micromirror Device

The primary spatial modulation element in this setup is the Texas Instruments DLP4710 DMD, a Micro-Electro-Mechanical Systems (MEMS) device [16]. Conventional wavefront shaping was historically conducted with phase-only Spatial Light Modulators (SLMs). In contrast, the DMD encoder used here translates binary patterns into spatially modulated optical fields, operating exclusively in the binary amplitude domain. Each micromirror independently tilts between two stable mechanical states (±12°) to direct incident light to sink on or off.

	Phase-only SLM	**Binary Amplitude DMD**	**Implications**
Modulation	Continuous 0-2π phase control	Binary ON/OFF modes	DMD has no phase information, must rely on intensity mapping
Hardware Integration	Requires extra, complicated, and sensitive hardware	HDMI input, consumer-grade with support for MATLAB	DMD is lower cost and is a simple setup
Optical Power	Lower due to composition	Higher efficiency	Laser power needs less added laser power

Table 5- Performance comparison between phase-only SLMs and binary amplitude DMDs

The scattering process conventionally is modelled from the superposition of multiple scattered waves with random phasing delays, captured from an SLM. Unlike the phase-only SLM used in classical wavefront shaping, the DMD operates exclusively in the binary amplitude domain, constraining wavefront control. Following the transmission matrix formulism established by Vellekoop and Mosk [9], the linearly modelled scattering process maps the unknown transformation seen, through Equation (1), at the camera plane at E_{out}, the relationship between it and the input field E_{in} and the incident scattering medium as **T**. This differs from intensity-only imaging, where the DMD is incapable of carrying phase information as a binary amplitude modulator. Provided with this, the DMD is an amplitude-only device, and the inverse mapping must be learned by Machine Learning, rather than computed through methods like phase conjugation. Which upon following the formalisms concerning amplitude-only modulation, the relationship between the displayed binary pattern M_{DMD} and the captured speckle intensity $I_{speckle}$ is expressed as the following equation,

2

$$I_{speckle} = \boldsymbol{S}\{M_{DMD}\}$$

where $\boldsymbol{S}$ represents the composite optical scattering operator of the system specific to the scattering diffuser lens. The non-linear appropriation of the propagation brought by the diffuser, summated by $\boldsymbol{S}$, is inherently modelled as linear due to the informetric measurement that is captured by the camera as a deterministic speckle field. This brings about the learned inference that embodies the workings of this project: where in finding the fixed mapping of $\boldsymbol{S}$, a linear exploit can be applied by Machine Learning to achieve control without explicitly requiring knowledge of phase information or **T**.

b) CMOS Camera

The ZWO ASI120MM Mini CMOS camera serves as the sensory component in the experimental system, acting as the objective observer of the captured speckle intensity. Unlike phase-sensitive optical detector hardware, such as Shack-Hartmann wavefront sensors [17] and interferometers, the CMOS camera used in this project portrays intensity-only feedback defined entirely outside the phase plane. This impacts data-driven control because the optimisation target of pattern reconstructive intelligent control will need to factor intensities as a variable, operating in a regime where the pixel values must remain strictly linear and stationary to ensure the mapping between the DMD states and captured intensities. Characterisation of camera response curves must factor in mitigating acquisition parameters with the aim of sufficient capture of

low-intensity interference fringes and high-intensity speckle peaks simultaneously. Maintaining operational gradient computation is a prerequisite to capturing a high-fidelity pattern. The significance the CMOS camera used in this project serves relates to accommodating to the constraints of its AR0130CS Mono Sensor, with gain amplification strategically calibrated to be applied above the sensor's read noise floor (4.0e~6.6e RMS[18]).

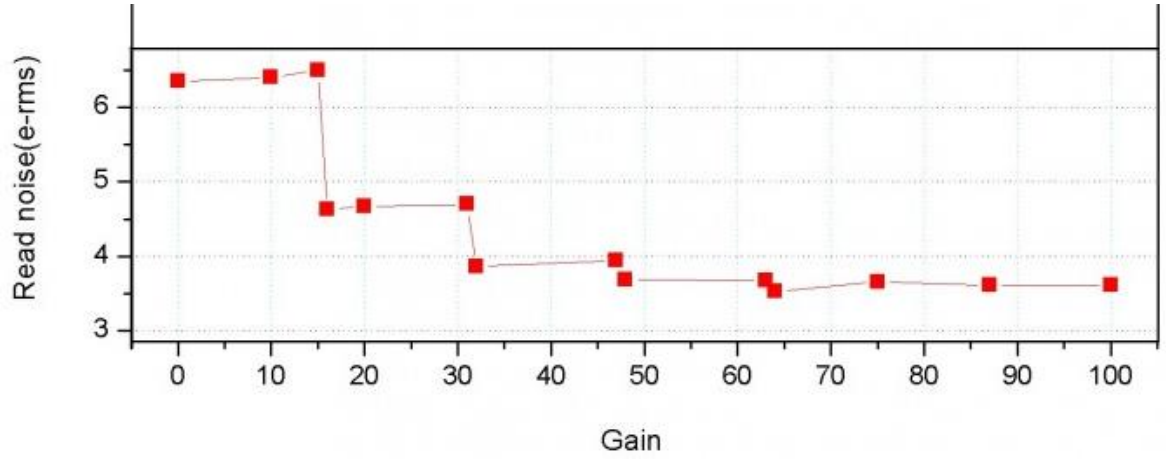


Figure 4- The read noise diagram for AR0130CS Mono Sensor with read noise(e-rms) and gain axes

Figure 4 illustrates an inherent trade-off between gain amplification and dynamic range, highlighting the role insufficient gain possesses in intensity maps by truncating values, introducing read and shot [18] noise post-integration onto the sensor. Coherent speckle formation is highly sensitive to sub-millisecond misalignment, highlighting the dynamic role both gain amplification and the deterministic characterisation of a CMOS sensor has on the input image quality. Another factor that alters the determinisation of the speckle fields is the exogenous illumination sources, such as stray light and monitor backlight, presenting itself in the environmental setup of the physical hardware as optical noise [19]. Mitigation of stray light is justified through variable additive background offsets stray light introduces, inflating the mean intensity $\langle \boldsymbol{I} \rangle$, resultantly depressing the speckle contrast $\mathbf{K}_s$ and distorting neural network intensity probability reasoning during feature extraction, decreasing learning merit and reconstruction accuracy. Mitigation of environmental noise regarding stray illumination is a process referred to as environmental grounding, which, with the view of preserving efficacy in the validity of this system is successfully achieved by supplementary optical baffling, a process of reducing stray light to a level at which it is tolerable [19]. A matte-black card enclosure custom cut to serve as a baffle ring, illustrated in Figure 5, is placed between the focusing optics and the camera aperture to minimise structured noise that would otherwise be misinterpreted as legitimate scattering features by neural networks.

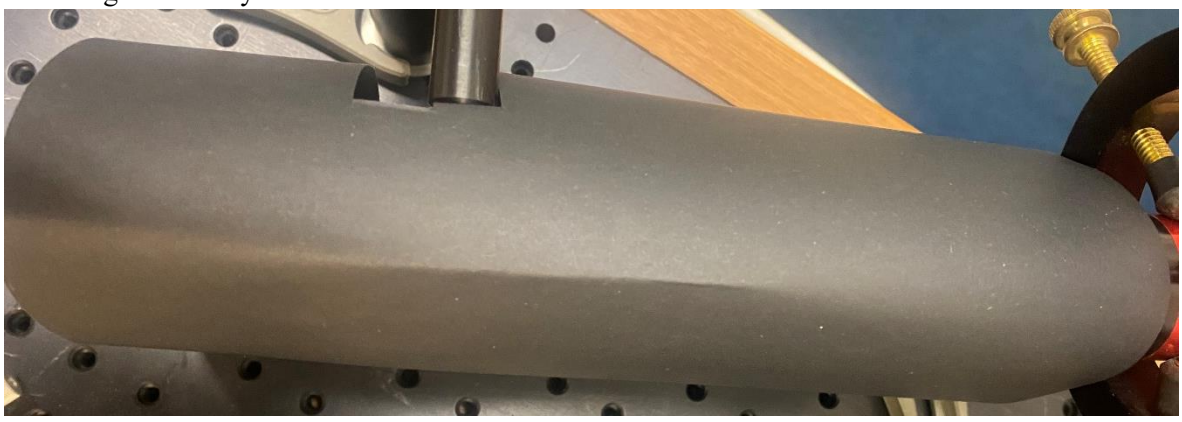


Figure 5- supplementary optical baffling and custom-cut matte-black enclosure

To quantify the linear response relationship of the input and output nodes, the training neural networks for the mapping task must scale proportionally, with numerical stability throughout the backpropagation of gradients remaining critically preserved. Brought about by min-max normalisation as a preprocessing function, pixel values in the integer range [0,255] to constrain input distribution within stable bounds. Normalising $I_{speckle}$ across the dataset effectively prevents the saturation of activation functions in deeper layers and avoiding the vanishing gradient problem, where gradients approach zero [20].

c) Simulation

An important step would be to understand the underlying dynamics that influence the optical system's interaction and feedback loop with the simulation environment itself. Before committing to the physical data acquisition process, there are controlled characterisations among the simulation setup that require modelling that would need to be evaluated to determine the optimal solution that will be utilised in this project. Coherent choices in simulation enables the process to isolate scattering operator $\boldsymbol{S}$, see section 2.1.1a) Digital Micromirror Device, for achieving intelligent control by considering confounding physical variables introduced through physical hardware like fluctuations in laser power or mechanical misalignment. An experimental approach without accounting or considering hardware characteristics would add unnecessary difficulties in the learning process of the simulation and would negatively impact the conclusions that would come from this project. The key objectives from the simulation of the optical system were:

Sim objective 1. Replicate the intensity statistics of fully developed speckle generated by the scattering diffuser under 520 nm illumination.
Sim objective 2. Generate a representative dataset of input-output pairs (**D2**) for a range of pattern types
Sim objective 3. Investigate the impact of camera read noise and laser power stability on the input
Sim objective 4. Generate validation data at the desired resolution for training of models to enable apt hyperparameter selection
Sim objective 5. Assess the expected signal to noise ratio (SNR) of the camera output under varying exposure and gain settings

Two initial options were considered for the simulation environment: MATLAB/Simulink and Python with OpenCV/PyTorch, with benefits summarised in Table 6.

MATLAB Advantages	Python Advantages
Accurate and existing code available for data acquisition	Extensive open-source libraries for custom implementation of Machine Learning features
Native support for hardware via Image Acquisition Toolbox and ZWO Camera Drivers	Vaster range of Machine Learning models, including community support
Pre-configured models found in the Integrated Deep Learning Toolbox available with adjustable parameters	

Table 6- MATLAB vs Python regarding simulation

While Table 6- MATLAB vs Python regarding simulation provides the viewpoint of system development trade-offs, with Python supporting a broader open-source ecosystem and greater flexibility in custom deep learning architectures, MATLAB's native toolboxes provide a modular simulation and control environment. MATLAB was chosen with the primary objective to leverage the robust, low-latency support for communication across the DMD and camera. Comparatively, choosing MATLAB/Simulink allows for the focus of the project to remain on intelligent speckle control, rather than hardware integration or low-level system debugging; there are added unpredictable runtime variables that a Python-based system would face, such as the need for third-party SDK wrappers [21] that would need to be installed for the camera to communicate, which would introduce inefficiencies in latency or miscellaneous problems unseen or unsupported in MATLAB.

For simulation objectives 1, 3, 4, and 5, the MATLAB environment was the most ideal to provide accurate solutions. The programmability and hardware integration can allow the Simulink model to be customised and launched within a MATLAB script, with Simulink being the primary simulation engine. Hardware integration provides the framework that enables the environment to automate initialising different DMD pattern geometries randomly, running them, alongside capturing and applying appropriate processing on the output. This highlights the simulation-to-application transition that will be achieved in this project. A seamless software architecture brings forth significant advantages in MATLAB being used for control code for the Machine Learning models, while also being the interface responsible for camera and DMD hardware control within one environment. Python offers a broad open-source selection of ML libraries, however the complexities of integrating hardware control impacts the scalability of the product, introducing an unnecessary production risk. The streamlined control MATLAB would have in the project system contrasts to the added uncertainties that a Python-based system would introduce. MATLAB lacks the need for Python bindings to a ZWO astronomical camera control interface that would need to be integrated [21] as a third-party software dependency.

2.1.2 Experimental Logic Flow

The diagram in Figure 6 indicates the operational sequence of the system and illustrates the translation of static optical architecture to Machine Learning solutions through sequential stages. The logic flow maps the data acquisition and Machine Learning model training pipeline with MATLAB, acting as the central orchestrator controlling the operation and timing of the logic flow. The main goal of the control software was to combine and synthesise the hardware control process into a repeatable and coherent pipeline, combining four distinct stages of production, allowing for the systematic The flow of operation surmises the dataset preparation, represented in Figure 6 as the first two (blue and green) quadrants, following the process of generating a paired dataset and processing it to formulate a learnable training dataset for quadrant three. To realise **D2**, a pattern set is simultaneously projected onto the DMD, with the resultant scattered speckle patterns captured and saved to storage.

The pattern set comprises block patterns with a maximum of 32x32 configurable binary macropixels, with the specification of the 10,000-pattern dataset further detailed in 2.2.3 Dataset Development. The final sequential step through the process of individual pattern capture is a 1 ms buffer period, the settle frame, that allows the camera a sufficient delay to capture every frame fully synchronised with the DMD broadcast function. This is enforced in simulation via setting a delay to the hardware-triggered function trigger(vid) [22]. The second, smaller dataset handling quadrant refers to the green preprocessing block, which handles the transition from **D2** to **D3** and **D4**. This quadrant coordinates the sanitation of data capture to provide a format most optimal to be directly ingested into the model architectures, involving the rectangular resolution of the capture being cropped to a square format, with the specification of these processes detailed in b) Data Preprocessing.

The latter two quadrants represent the actuation of the project goal, incorporating the training and assessment of the trained models. Datasets flow directly from the previous quadrants and are partitioned deterministically using a fixed random seed (see a) Data Acquisition) to ensure training reproducibility.

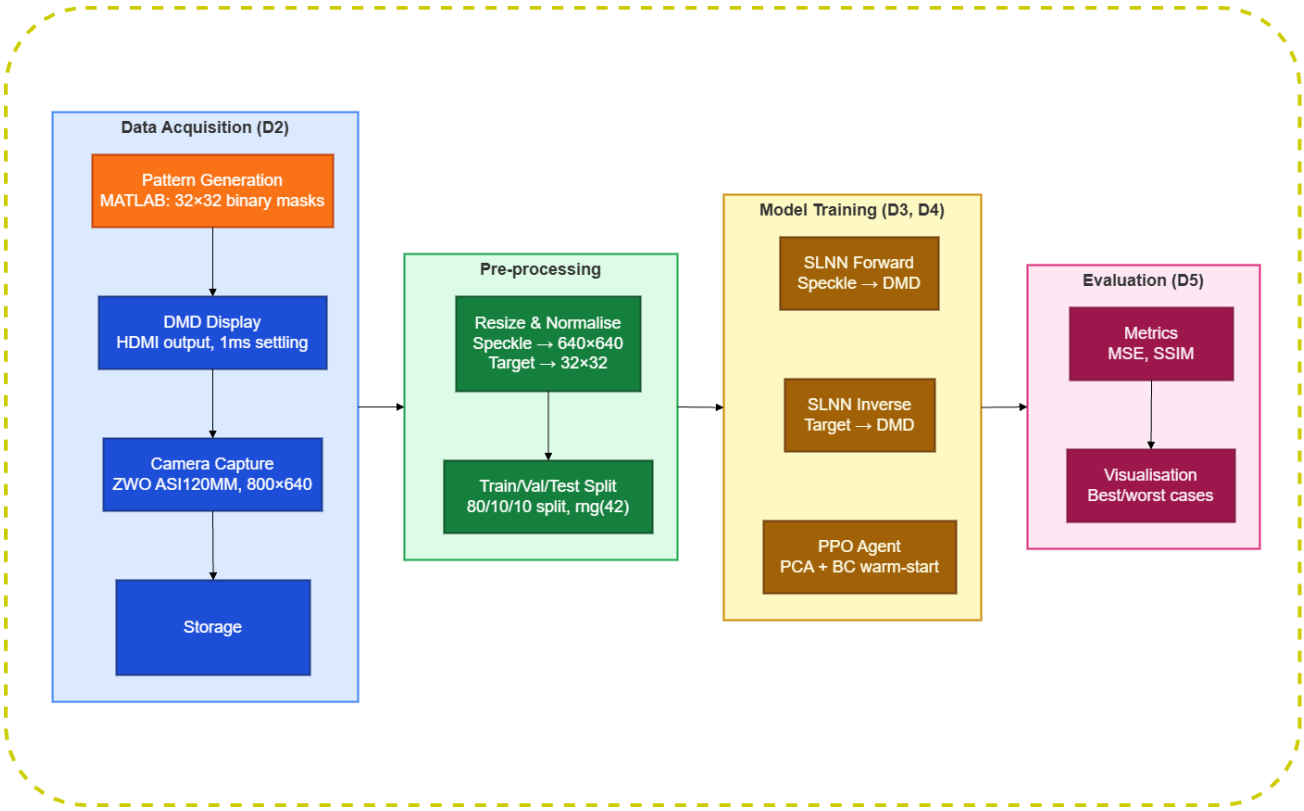


Figure 6- Experimental Logic Flow Diagram

2.2 System Development (Realisation and Sub-System Testing)

2.2.1 Machine Learning Model Architecture

The architectural design of the two Machine Learning paradigms discussed in this project relate to the development of methods used to address the fundamental mapping of Equation (2), where $\boldsymbol{S}$ represents the composite scattering given unto the image by the scattering diffuser, leading to the reverse mapping operated to isolate Equation (3):

3

$$M_{\mathrm{DMD}} = \boldsymbol{S}^{-1} \{I_{\mathrm{speckle}}\}$$

Without using computationally intractable methods, such as direct mathematical computations of a physical representation of the complex transmission matrix **T**, a data driven response from the **MLP** and **PPO** agents illicit an approximation of the inverse of the diffusive material $\boldsymbol{S}^{-1}$ through rigorous training. This approach successfully leverages the fact that Machine Learning is fully capable of handling the complexities resulting from the presence of vast amounts of imaging data [7] that are otherwise highly challenging to process.

The **MLP** model implemented in this project does not solve the optical scattering problem with mathematical inversions and calculations, but through a learning period in which it determines an implicit and appropriate mapping. This process correlates paired examples of I_{speckle}, shown as the speckled capture imagery, and M_{DMD}, the DMD projection pattern known as the ground truth. The **MLP** model's architecture can be broken up into two processes, run sequentially in the training paradigm, both learning to predict the DMD pattern output given a captured speckled input. Fundamentally, the **MLP** model is an example of a neural network, a layered adaptive system where a series of layers work as an adaptive system that learns from data to recognise a specified pattern [23], matching the specification of the neural network chosen for this project.

The first process relating to the training of the **MLP** model relates to the forward model as seen in Figure 7, which is trained to learn to predict the DMD pattern given a captured speckle input image as pattern reconstruction. The input is a frame that is sized at 64x64 pixels, in which frames must be flattened into a vector, describes as d_{in}, where d_{in}=4096 intensity values, representing the sum of all pixels in the input frame. The process the network undergoes relates to the act of dimensional reduction of progressively extracting and compressing optical features it can infer to be relevant.

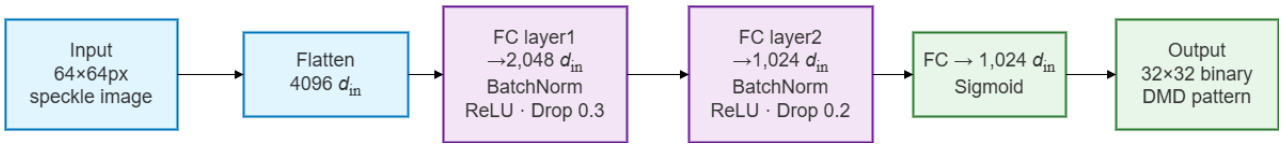


Figure 7- Forward model schematic

Figure 7 broadcasts compacting layers, which systematically scale down the number of d_{in} parameters per layer, with two hidden, fully connected (FC) layers reducing the high-dimensional pixel input to representational features. The first FC layer, hidden layer 1, performs the initial abstraction process and reduces the dimension of d_{in} from 4096 to 2,048, and achieves these compacting processes through three operations. The first hidden layer uses a ReLU activation, defined as an activation layer which passes positive signals and limits negative signals to zero, leading to help the layers learn the complex relationships between inputs.

There is a regularisation step in the form of batch normalisation, which is a hyperparameter that involves standardising layer inputs to stabilise the internal distribution of layer activations. Figure 7 attributes a dropout function to the hidden layers, which is the rate of randomly disabled connections during training to prevent overfitting to specific data modes, referred to as the dropout rate. This rate transfers from 30% of connections in the first hidden layer to 20% between layers, alongside further abstraction to 1,024 inputs, which are ultimately converted through a sigmoid function to the 32x32 output pattern.

The second similar process is an inverse controller model seen in Figure 8, which complementarily solves for pattern reconstruction with the idea that thin diffusers are approximately linear in light intensity relation, altering a similar architecture to accept an intensity distribution at 96×96 pixel input resolution to a 32x32 output, and being intentionally more shallow with a single connected layer to empirically evaluate the linearities between the broadcast 96×96 DMD image and the intended pattern. During the training where the model infers relationships, the single layer allows for direct mapping of the 1024 d_{in} physical output and correlates directly with further defined specifications in ii. MLP Hyperparameter Specification and Comparison.

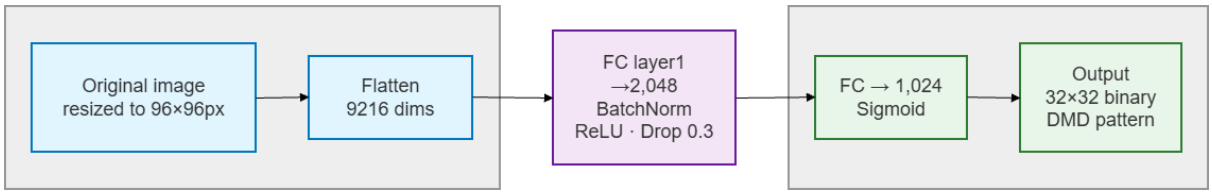


Figure 8- Inverse controller sub-model architecture

Initially, similar architectures available in the native MATLAB environment were considered, with a U-Net incorporating a skip-connection preservation of high-frequency speckle correlations [24] being a feature that would provide an ideal appeal for pixel reconstruction. Li et al. [25] evaluated multiple U-Net variants against clinical datasets and determined that architectures with enhanced skip connections achieve higher accuracy on trained data over baseline designs.

However, this was assessed against the severe computational implications it would pose. Through the limitations seen in 2.1.1 Experimental Setup, any segmentation performance gains attributed to the U-Net would be hindered massively by the additional computational overhead [24] introduced by the U-Net framework. Such added complexities introduced by the network would otherwise ideally be mitigated by processor power, but this project is conducted with a low-power portable CPU, and operation under excessive computational load is only feasible with a dedicated GPU. Proceeding with a U-Net would therefore prevent delivery of the D8 deliverable in 1.3 Aim and Specification.

The **PPO** agent is the second chosen Machine Learning model, which reframes the system architecturally as a reinforcement learning paradigm, where an agent, would refine the DMD control pattern to maximise a cumulative reward following a trial-and-error learning process known as the Markov Decision Process (MDP) [26]. Hitherto, would effectively result in a learning function seen in Equation (4), where π_θ relates to the learning policy [27]:

$$\pi_\theta \approx \boldsymbol{S}^{-1}$$

Figure 9 defines the process in which the raw inputs are made tractable by projecting them onto a 256-dimensional space, using a Principal Component Analysis (PCA) compression function to fit exclusively on the training dataset to prevent the model overfitting against unseen data. The second PCA compression objective is to limit the trainable dimensions the actor network interacts with from 4,096 to 256, exceeding memory capacity allocation allowed in the specified computational hardware. The next action that defines Figure 9's blue function is Behaviour Cloning (BC), which pre-trains the system using known examples so it can then begin moving away from trial and error and into learning a sensible training strategy, reducing added time for the model to learn in training through 20 epochs. Moving towards the fine-tuning partition of the model, noted as the right side of Figure 9, the central actor outputs a continuous Gaussian policy $\pi\theta(a|s)$ [27], where a is equal to the DMD control pattern and s represents the PCA-compressed speckle observation, to compute probability values and convert continuous outputs to pattern states, with a Critic function estimating the expected reward to stabilise the estimation values and policy updates. The second part of the fine tuning has the agent interacting with the optical environment, reconstructing target DMD pattern based on a reward policy, $R_t = -\frac{1}{5}\text{MSE}(M_{\text{Predicted}}, M_{\text{DMD}}) + 0.02\text{xHD}$, where HD is the Hamming distance and $M_{\text{predicted}}$ is the predicted pattern, enabling wavefront shaping inversely with further defined specifications in section 2.2.4b.

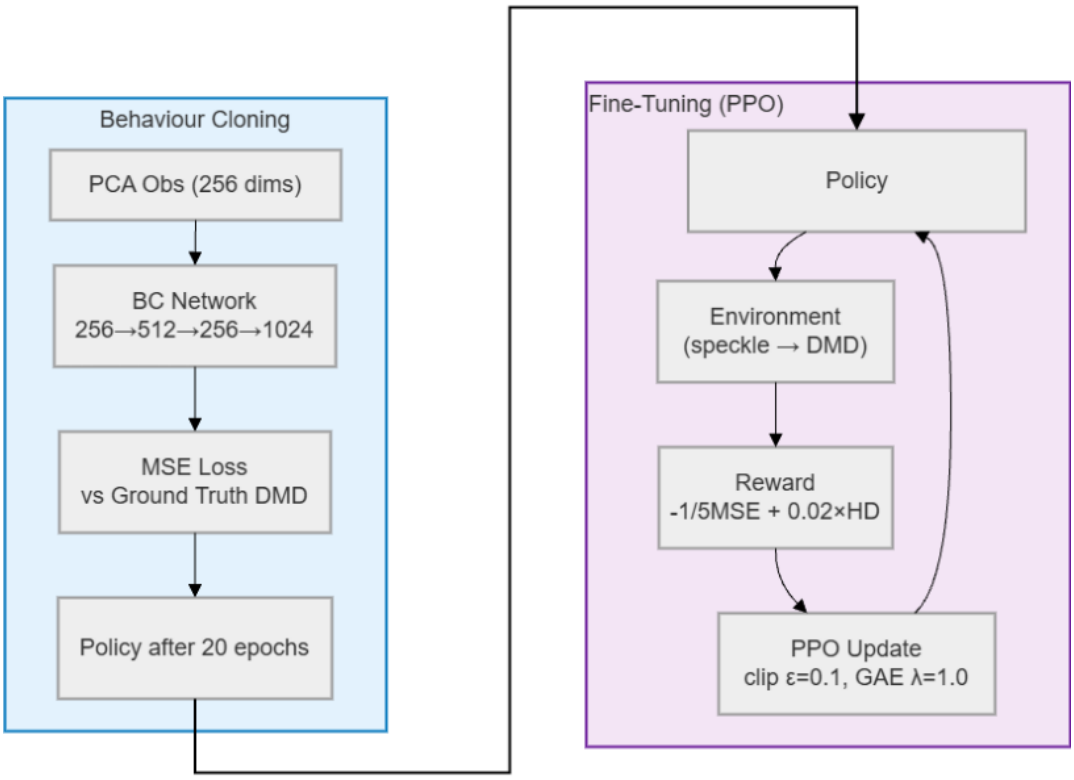


*Figure 9- **PPO** model schematic*

Both architectures are modelled to be equivalent, to assess sole architecture and control strategy suitability, with both sharing the objective of learning the inverse mapping S^{-1}. The implementation of the Supervised Learning and Reinforcement Learning are closely integrated in a shared MATLAB environment, ensuring the equal comparison in 3.4 Comparative Analysis is indicative of architectural choices alone.

2.2.2 Experimental Equipment Calibration

The optical system described in 2.1.1 Experimental Setup follows the design made by Mr. Gabriel Pacheco [14], adapting the process of integration of the engineering principles and hardware synchronisation to create a reproducible dataset for the Machine Learning models. it is made to generate fully developed speckle input patterns and would involve the optical hardware setup.

2.2.2.1 Active Region Analysis

The initial calibration stage would establish the connectivity of the DMD projection, and the transformation captured on the ZWO Camera. Following validation processes adapted from a prior intelligent control implementation by Mr. Jialu Zhu [28], where a high contrast bullseye projection pattern seen in Figure 10a, is broadcast to the DMD for the purpose of verifying spatial alignment with the cameras capture field. Its concentric properties serve as a geometric reference feature for recognising and capturing spatial consistencies and its symmetry would serve as an optimal analysis for any focal misalignment seen from lens positioning. Figure 10a is used to analyse the intensity profiling of pixels across the camera field and to reveal the regions where intensity is the most consistent, to streamline the data training process.

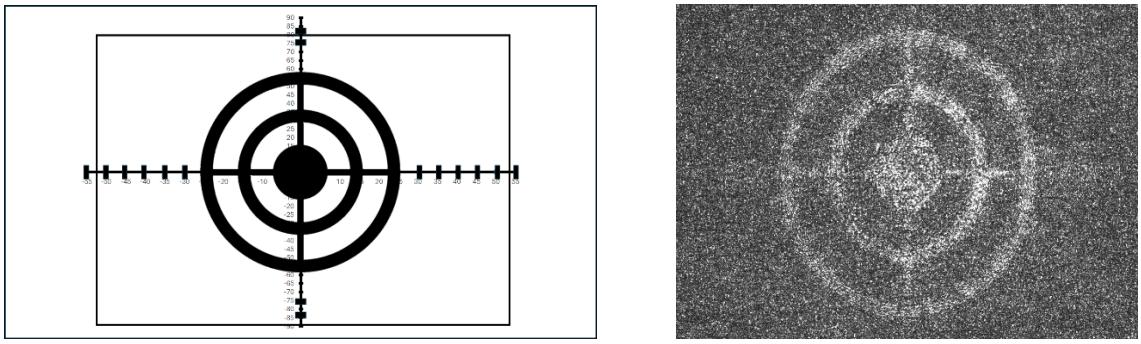


*Figure 10- **(a)** Bullseye image generated in Microsoft PowerPoint displayed on DMD, **(b)** Corresponding output captured by ZWO camera displayed in ASICap*

Studying visually, the intensity profiling shown from the image at Figure 10b, captured from the camera's native camera capture software ASICap [18] (under automatic settings), can be described as preserving most of the information inside the outermost bullseye circle. The central axes diminish in intensity outside the centre, and further empirical analysis confirms that the region with the most stable and therefore suitable ranges of intensity are confined to the square region within the outline of the circle meaning that the active region recognised for this project would deal with projections of DMD patterns at a 528x528 pixel region, representing the nearest square region for spatial sampling. The significance of a square region is explored in b) Data Preprocessing.

Limiting the DMD pattern of a maximum 1920x1080 pixel native resolution to 528x528 projection area presents its significance as defining hardware limitations of the project, and the active region choices highlight key mitigation strategies chosen to stabilise a consistent signal-to-noise ratio, constraining data capture to focus on the pixels of the capture that have the capacity to display the necessary pixel intensity ranges that can be discriminated by the Machine Learning models for inference. Determination of this 528x528 value relates to the hardware setup and the limitations of specific components that are employed in the experimental setup.

Resultantly, to reverse the notion the 528x528 projection the DMD is the only valid regions, to calculate what resolution range to configure the camera's contiguous active zone is to fit the region to a pre-existing camera resolution setting native to the specification of the camera. Capturing the active region in its entirety is closest to the corresponding configured setting of an acquisition resolution of 800x640 pixels, which can then be cropped into a square 640x640 to guarantee 1:1 pattern-frame mapping integrity prior to dataset acquisition. This finalises in the 528x528 projection area specified mapping to the 800x640 camera capture region, which, after further preprocessing ensures a 1:1 aspect ratio mapping between the DMD macropixels and camera capture pixels.

2.2.2.2 Camera Configuration

An objective of data acquisition is to find an optimal value in which the sensor of the ZWO camera interacts with the MATLAB environment, with the MATLAB interface employing separate values for exposure and gain, see section 2.1.2. It is critical to find a value that allows for the optical field to be accurately represented by the camera capture without sensor saturation or indistinguishable pixel intensity representation, diminishing the learning ability of the Machine Learning models. To achieve a sufficient value for the exposure levels, a subsystem test, visually synthesising different exposure-gain pairs was conducted, acquired via Code Snippet 1, realised in Figure 11.

MATLAB uses the Image Acquisition Toolbox as a bridge to interact with the ZWO camera, and uses the winvideo interface, which is controlled on a logarithmic scale [29] with the subsystem implemented testing a range of exposure levels to empirically determine an optimal exposure level. Following the bullseye image in section 2.2.2.1, the subsystem is modelled to ultimately assess 5 different exposure levels, $E\{$-8.8, -7.3, -6.8,

-2.3, 2.8}, with each level of *E* fixed at the analogue gain of 10.3 as a consistent baseline. The camera would acquire a single frame after settling to the new parameters in a 1 second period before capturing and saving the image, liberally using x10 the settling period of the data acquisition algorithm. To maximise the coherent signal-to-noise ratio, the final exposure level would need to be the best suited configuration to capture the patterns' full dynamic range.

Figure 11 reveals the sensitivity of the log configurations around (a), (b), (d) and (e), with high intensity saturation in overexposed regions. Low exposure seen in b and a result in poor capture visibility, whereas values of intensity in (d) and (e) show that the sensor is saturating, causing information to be completely incoherent with values tending towards 255 of the [0,255] intensity scale, exhibiting poor balance in range. The optimal region is (c), providing a well distributed pixel intensity profile for model inference, maximising optical fidelity and allowing for a balanced dynamic range of intensities.

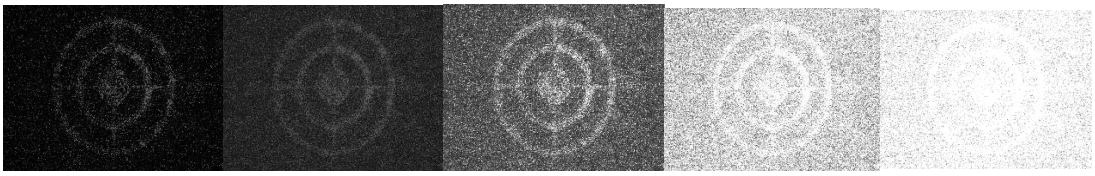


*Figure 11- **(a)-(e)** Captured exposure calibration with E{-8.8, -7.3, -6.8, -2.3, 2.8} respectively*

2.2.2.3 Scattering Diffuser Analysis

Using the calibration configurations in sections 2.2.2.1 and 2.2.2.2, a single frame speckle contrast measurement could now be applied on the diffuser to analyse the characteristics of the chosen scattering diffuser to establish a quantitative figure of the learning difficulty in which the Machine Learning models will be operating under. Speckle contrast refers to, determining the distance between the ratio of intensity fluctuation to the average brightness of a speckle pattern. Following Goodman's statistical theory of fully developed speckle [1], there is a learned relationship between an image's spatial standard deviation of pixel intensities, referred to σ_I and its spatial mean pixel intensity represented as $\langle I \rangle$. A perfectly monochromatic pattern of speckle ideally would have a speckle contrast of $\mathbf{K}_s$=1.0 [2], where the value of 1.0 represents an image that has maximum contrast. Computing the value of speckle contrast ($\mathbf{K}_s$) results in equation (5):

$$\mathbf{K}_s = \frac{\sigma I}{\langle I \rangle}$$

where σ_I refers to the standard deviation across the frame and the $\langle I \rangle$ represents the averaged mean intensity. Calculating the diffuser's strength by averaging the speckle contrast values in every pixel informs subsequently of the level of difficulty that optical learning algorithms would face when reconstructing the original DMD pattern given said speckle as an input, forced under the conditional bounds of $0 \leq \mathbf{K}_s \leq 1$.

Figure 12 shows the speckle intensity map of a speckle field when a captured reference frame of an all-off mirror configured on the DMD, where higher values are represented by brighter colours in the thermal representation map. Speckle contrast is more prominent in the thermal heatmap in more dense speckled regions, as the glass diffuser is non-linear in its entire surface resulting in the top left of the sensor area having warmer values of speckle contrast. Computing the matrix value of $\mathbf{K}_s$, averaged across the full 640x640 sensor resolution yielded a mean speckle contrast of $\mathbf{K}_s = 0.5046$, suggesting the moderately strong nature of the scatterer, where extremely strong diffusers would yield a lower value where $\mathbf{K}_s < 0.4$, and lighter diffusers would yield higher contrast values of $\mathbf{K}_s > 0.7$, where the found value of Figure 12 impacting the experiment as it can be evaluated under the baseline difficulty and analysed in the results that are found through the learning process.

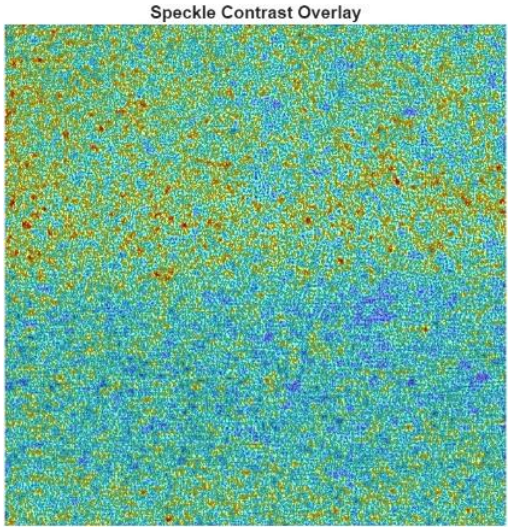


Figure 12- Speckle contrast overlay of an all-off speckle pattern

2.2.3 Dataset Development

a) Data Acquisition

The process of data acquisition is categorised into to an assorted pattern dataset where dimensionality of the active region control is reduced from induvial pixels to 1,024 grouped control channels as grouping the DMD active resolution of 528x528 as a cluster of pixels, grouping the 278,784 pixels as 32x32 macropixels, where each pixel is clustered as a unified binary actuator, with each aggregating a block of roughly 272 physical mirrors via nearest-neighbour interpolation run in MATLAB. The 32x32 macropixel resolution indicates the 1024 independent control channels in which the Machine Learning models can operate and define a tractable space for pattern reconstruction, allowing for the reconstruction of the DMD pattern to be achieved intelligently without iterative refinement. Following Turpin et al. [12], the decision to employ grouped macropixels was made to make the learning process tractable computationally, with a lower resolution correlating with a lesser total parameter count for the Machine Learning models, with no clustering of pixels resulting in $> 2x10^6$ variables, an infeasible amount of control channels. However, a major discrepancy when assessing the transferability of the Turpin et al. [12] methodology in regards to dataset design is that their experimental parameters are not confined to consumer-grade hardware, leveraging processing power magnitudes greater than this project's CPU-only constraint to train against 64×64 macropixel patterns representing 4,096 independent nodes that would otherwise be intractable considering the hardware conditions employed here. Resultantly, A reduced macropixel resolution with 32x32 configurable nodes was adopted to account for the disparity between computational capacity in both projects, maximising the spatial coverage afforded by the experimental setup.

	Pattern Type	**Resolution**	**Quantity**	**% of Total**
a	4x4 random binary	4x4 macropixels	3498	35%
b	All-on/All-off reference frames	Full panel	2	0.02%
c	Hadamard basis patterns	32x32 (degrees of freedom)	1024	10.2%
d	Sparse random binary	32x32	1000	10%
e	Circular focal spots	32x32	1000	10%
f	Stripe/grating patterns	32x32, 0°/45°/90°/135°	1000	10%
g	Quadrant/block patterns	32x32	1000	10%
h	Ring/annular patterns	32x32	1000	10%
i	4×4 Validation RBP	4x4 macropixel	476	4.76%
	Total		**10,000**	

Table 7- Composition of Dataset and Class Distribution

The dataset is made up of 10,000 unique pattern pairs, with 9 types of patterns offering distinct spatial makeups, randomised with the random seed rng(42) in MATLAB to ensure it could be reproduced. The dataset, compartmentalised in Table 7, is engineered to be stratified into the 9 individual patten classes to provide a unique distribution of random inputs for structured acquisition onto the learning dataset for the goal of creating a robust, statistically representative learning paradigm. Roughly 35% of the dataset follows the convention of a course 4x4 resolution pseudorandom binary grid to initialise the fundamental speckle to DMD pattern configuration mapping learned by the Machine Learning algorithms. It provides an easier learned inference, and its 2^{16} (65,536) possible permutations prevent memorisation and a repeatable benchmark for testing unseen pattern evaluation.

Including Hadamard basis patterns at a 32x32 grid provides a unique covering of the DMD's active area. Hadamard patterns are 2D basis patterns which form a complete orthogonal set [30] where orthogonal refers to the distinct non-correlation across the patterns, allowing for independent extraction of spatial information for all modes in the DMD's region. Hadamard patterns are individually mutually orthogonal, meaning that each unique pattern probes the scattering diffuser in a mathematically independent matter. The systematic operation used to determine the matrix $\boldsymbol{H}$ would operate to resolve all permutations without redundancy or repeated patterns to a limit of the quantity 1,024, representing the degrees of freedom of unique patterns totalling the sum of any given resolution .The resolution chosen corresponds to the smallest controllable macropixel value, where there is a limit of (binary) 2^{32x32} possible binary configurations , with the mathematical algorithm resolving Hadamard matrix $\boldsymbol{H}$ consisting of only +1 and -1 matrix values [30], Hadamard patterns are inherently binary patterns.

The remainder of the dataset is allocated across specialised classes that provide a diverse spatial set where the learning algorithms can ascertain different optical laws and flexible arrangements of any permeation of the 32x32 grid. Pictured in Figure 13(d)-(i), validate the adaptability by allowing pairings that will simulate constructive inference at target positions across the DMD active view. The isolated 476-pattern subset, Figure 13(i) is generated with an independent seeding, rng(137), to further prevent overfitting towards previous seeding and provide a robust generalisation test.

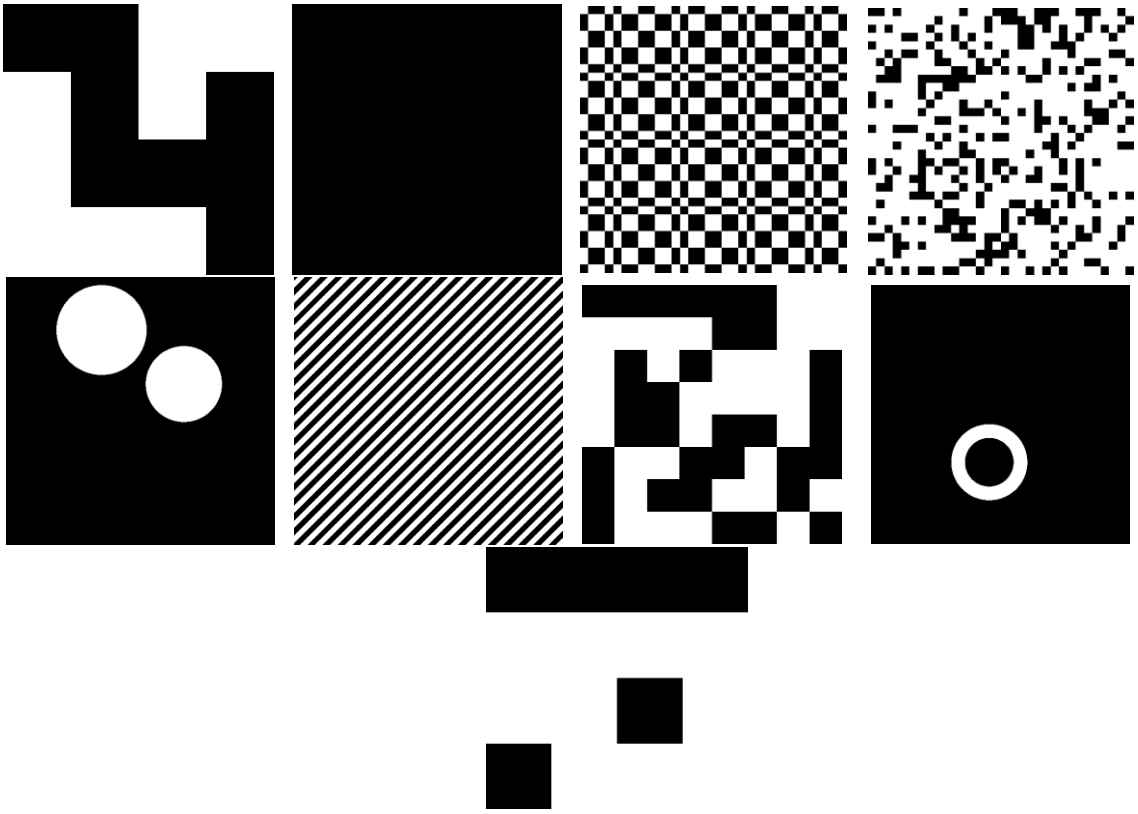


*Figure 13- **(a)-(i)** Representative samples from each of the stratified pattern classes (left-to-right) corresponding to Table 7.*

b) Data Preprocessing

Raw camera frames captured in MATLAB undergo a three-stage workflow to resolve hardware discrepancies that will need to be resolved to bridge the gap between raw capture and processed dataset. Post-acquisition, issues such as quantifiable input latency, non-square resolution and optical inversion exist in every captured frame, leading to misalignment that will lead to incorrect learning inference.
The ZWO ASI120MM Mini observably exhibits computed trigger latency, and capture of the input frames resultantly include a null frame at i=1, where i defines the pattern index in the sequence. Fulfilling deliverable goal **D2** makes it imperative to preserve proper 1:1 pairing between inputs and outputs, lest a knock-on temporal misalignment affects the suitability of learned mapping. The preprocessing workflow captured in Code Snippets 2-4 is illustrated in Figure 14 through the transformation in Equation (6):

$$I_{\text{aligned}}(i) \leftarrow I_{\text{raw}}(i + 1)$$

where I_{aligned} and I_{raw} represent indexed frames on the captured sequence.

After alignment is achieved, observation confirmed that an inversion on the vertical axis was observed due to the capture architecture in MATLAB image acquisition toolbox. Compensating for this involves an individual application of the flipud function broadcast in Code Snippet 3. Orientation correction is also validated across the dataset, using the Pearson correlation coefficient to check coordinate consistencies. It is validated through the Pearson value $\mathbf{r} > 0.98$ as the near perfect alignment of the flipped frames I_{flip} against the expected reference pattern, resulting in the quantifiable process being demonstrated in Equation (7):

$$I_{\text{flip}}(Y_2{:}Y_1, X_2{:}X_1) = I(Y_1{:}Y_2, X_1{:}X_2)$$

Thirdly, the native 800x640 capture contains the excess blank margins that would be redundant and only complicate the mappings of peripheral pixels to the output. A basic crop of 80+80 pixels on either side

ensures that the 640x640 square resolution can be input at uniform isotropic square frames, ensuring efficient feature extraction processing for the Machine Learning models.

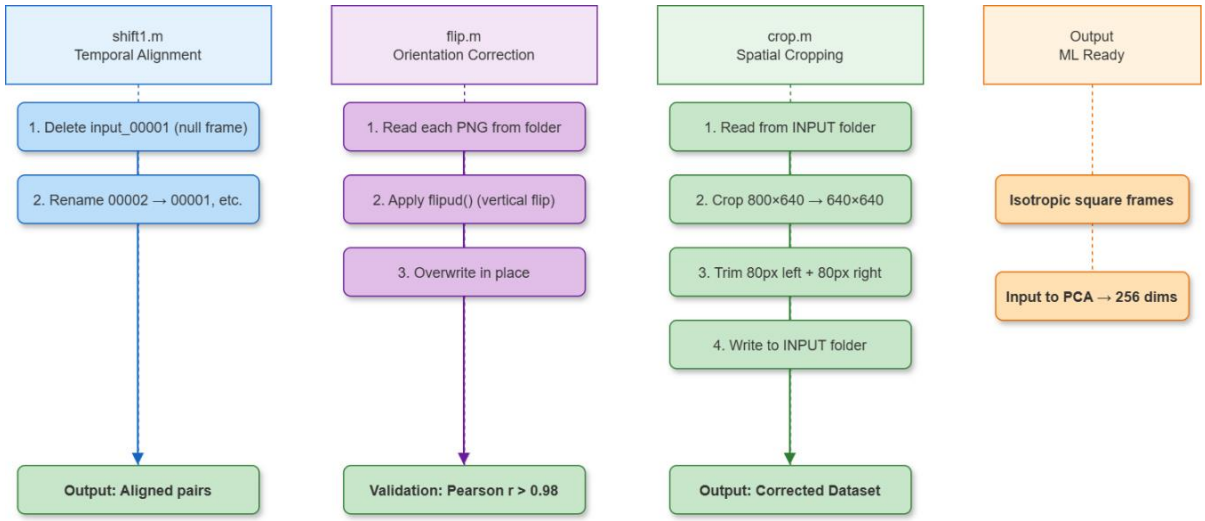


Figure 14- Representation of preprocessing pipeline through three MATLAB scripts

c) Data Normalisation

Resulting input-output pairs following preprocessing follow an 8-bit unsigned integer format represented as raw intensity matrices $I_{raw}(x,y)$ as discreet integer pairs [0,255] in the intensity domain. Feeding these intensity values in the Machine Learning models would introduce instability and imbalances due to weighting distribution being skewed by larger values. As a result, the correct implementation of normalisation would be to apply min-max scaling, with the transformation being represented in the Equation (8):

$$I_{\text{normalised}}(x,y) = \frac{Iraw(x,y)}{255}$$

where the process is justified in c) Data Normalisation. Division by 255 preserves the linear relationship within the input, allowing for the mapping of all intensity features to be shown as a floating-point value in the domain of [0,1], ensuring numerical stability.

Function	Operation	Rationale
if exist(filepath, 'file')	File check	Prevents coding errors if a file is missing and logs a warning.
imread()	Image reading	Loads the PNG file into a MATLAB array format to allow for conversions into the single-channel grayscale input.
imresize()	Geometric resize	Matches the expected input dimensions in any case it isn't already in the required format.
single(img) / 255.0	Normalisation function	Equation (8) that preserves relative intensity scaling.
[target_size]	Output	Optupts file in a fixed-dimension format compatible with correct formatting rationale.

Table 8- Normalisation workflow

Table 8 demonstrates key functions undertaken in the individual Machine Learning model architectures determining the sequence of reading and converting the input files into intensity-based normalised matrices. Min-Max scaling imposes minimal computational time complexity expenditure (**O**(1) per pixel) to achieve appropriate dataset-wide consistencies.

2.2.4 Machine Learning Development

a) MLP Code Development

i. MATLAB Model Implementation

The **MLP** architecture is an implementation of a feedforward neural network to map the inverse scattering map between the captured speckle pattens and the DMD patterns to binary states. This model is made up of two sub-models that are used to measure different methodologies of achieving a neural network implementation of intelligent control, with the same pattern reconstruction inference being obtained from the forward model and inverse model, see 2.2.1 Machine Learning Model Architecture.

Implementation of the model, seen in Code Snippet 5 is structured to produce 2 distinct models, that can further be tested in the final system Testing to benchmark comparatively. Memory management, high dimensionality of input and computational constraints would resultantly create boundaries and limitations of use, needing concise choices in design and consideration when analysing results. Table 9 describes the specifications of the layers that comprise the forward model (also illustrated in Figure 7) and its training strategy, addressing areas such as high dimensionality of the input being abstracted through downsampling and batch normalisation layers being applied to prevent gradient instability from increasing to an unsuitable amount. The goal of these layers are to make sure there is a stable structure during the model's Backpropagation and considering that my hardware specifications, see section 2.1.1, limits my Machine Learning models to operate via the CPU, the model implemented is lightweight compared to contemporary counterparts that utilise deeper convolutional models. The dimensionality of utilising the 4096-pixel input means that the first fully connected layer contains 8.4M parameters, with parameters referring to the overall amount of trainable neurons and biases in layers, and the value being determined by the product of the input and output neurons plus miscellaneous bias terms, with the limitations of a less computationally powerful setup, it is necessary to carefully facilitate a reduced layer size to prevent memory overuse during training. Batch normalisation applied after the fully connected layer and before every ReLU accounts for the shift in distribution of inputs during training, limiting the amount of extreme gradient values and affecting the computational efficiency by reducing the overhead, a process that accelerates training convergence, with the specification shown in Table 9, it is designed with default momentum, meaning that running mean updates. It should be known that the network prioritises converging with stabilised activation steps. After both layer functions, a ReLU layer acts as the activation to enable the network to learn complex relational mappings and is chosen to avoid the vanishing gradient problem common to sigmoid networks [20] [31]while maintaining viable computational speed. The neurons a Machine Learning architecture employs in any given layer affect the expressive capacity of the model, mitigating underfitting risks, with the rate of neurons per layer diminishing until it can be mapped one-to-one with each pixel on the output prediction matrix, allowing for the sigmoid function seen in the sigmoidLayer, with sigmoid outputs representing the probability of weighted neurons turning ON, to be applied directly before the regression layer, enabling loss functions to calculate analytical gradients at every epoch.

A main engineering bottleneck arises from the trade-off between the total amount of epochs used in training and the total processing time. Epochs are extremely critical parameters as they define the training exposure, and are expected to minimise loss, with optimal paradigms allowing for convergence of learning inferences to be achieved before the model overfits.

Layer Sequence	Configuration	Rationale
featureInputLayer	4096 neurons, Normalisation: none	Fully accepts 64x64 input after 0.5 scale image resizing step
fullyConnectedLayer	2,048 neurons	Initial compression of neurons
batchNormalizationLayer	Default momentum	Stabilises gradients in epochs

reluLayer	Name: relu1	Required non-linearity across spatial features
dropoutLayer	0.3 probability	Mitigate overfitting
fullyConnectedLayer	1,024 neurons	Mapping to DMD space dimensionally
batchNormalizationLayer	Default momentum	Secondary stabilisation of gradients during training
reluLayer	Name: relu2	Secondary non-linearity across deep features
dropoutLayer	0.2 probability	Secondary overfitting mitigation
fullyConnectedLayer	1,024 neurons	Output dimension matching to match DMD grid
sigmoidLayer	Output [0,1]	Maps weighting across models to pattern probabilities
regressionLayer	MSE Loss	Loss function to optimise intensity deviation.

*Table 9- **MLP** Model layer technical specification*

ii. MLP Hyperparameter Specification and Comparison

To rationalise the selection of epochs used in this forward model, comparisons between hyperparameters were conducted, notably the Optimiser, Learning Rate and Batch Size, all, alongside epochs, affecting overall generalisation errors seen with data. To maximise the forward model's learning efficiency, the model utilises the Adam optimiser with a learning rate of $1x10^{-4}$, as it updates parameters dynamically with the optimiser working with scaling decision-making in estimation steps. The relevance of the learning rate regarding the optimiser affects gradient step sizes, with the Adam Optimiser enabling adaptive learning rates to adjust per parameter, improving overall training efficiency. Another competing running optimiser considered, was Standard SGD, however, it didn't suit my learning specifications as it is seen to frequently got stuck in training modes with imaging. Another roadwork that needed to be navigated with my relatively lower processing power hardware constraints is modifying the Batch Size to optimise its memory efficiency for my Hardware's memory allocations at a safe level.

Hyperparameter	**Attribute**	**Rationale**
Optimiser	Adam	Per parameter learning rate for speckle features
Learning Rate	$1x10^{-4}$	Prevents gradient explosion in layers
Max Epochs	30	Sufficient for convergence to stable loss without overtraining
Mini-Batch Size	128	Fits specification CPU memory sufficiently
Shuffle	Every epoch	Eliminates ordering biases
Validation Frequency	30 iterations	Match epoch length for analysis per epoch
Validation Type	Holdout	Independent validation split

*Table 10- **MLP** Training Hyperparameters configuration*

The inverse model, with the abstract modelling shown in Figure 8, differs quite a bit with the forward model, considering that it is intentionally shallower, with a single hidden FC layer. The inverse model has 1.8x more Parameters, trading the layer depth for the wider width, accounting for the hardware employed not being able to support a modest amount layer, which is normally suited for spatial image processing. Having considered the implications of the rationale for Mean Squared Error (MSE) as a loss function, the loss function is embedded in the automated training pipelines, with it having the significance of penalising large errors. The loss function, supervising the model, follows an automated two-stage evaluation in the systems, with it both being used to update parameters. During the training phases, the loss function is built into layers, such as the regressionLayer to automate backpropagation calculations, instead of computing losses manually each iteration.

Specification	Difference	Rationale
Input Resolution	64x64 vs 96×96	Downsampling applied to denser forward model can be reduced in inverse controller due to lower hidden layer count.
Input Layer Dimensionality	4096 vs 9216	Resultant total input pixel count, with the inverse model accepting a higher dimensionality due to its direct mapping with more features and higher initial spatial resolution.
Hidden Fully Connected Layers	2 vs 1	Intentionally shallower to assess a more linear approach between input and output given higher initial feature count.
Total Trainable Parameters	Approx. 11.5M vs 21M	Trade-off between layer count and individual layer width.

Table 11- Specification differences between Forward model (former) and Inverse Model (latter)

Layer Sequence	MATLAB Function	Configuration	Name	Rationale
Input	featureInputLayer	4096, 'Normalization','none'	'speckle_in'	Accepts the flattened 64×64 speckle frame (4,096 intensity values) after min-max normalisation to [0,1]. No input normalisation applied, preprocessing handled upstream.
Hidden 1	fullyConnectedLayer	2048	'fc1'	Initial compression from 4,096-dimensional raw speckle space to 2,048 learned features. This layer contains 4,096 × 2,048 + 2,048 = 8,390,656 trainable parameters. The width was constrained by the 32 GB CPU RAM limit; deeper or wider layers caused out-of-memory errors during training.
Normalisation 1	batchNormalizationLayer	,	'bn1'	Normalises activations across the mini batch before the ReLU activation. Stabilises gradient flow by maintaining zero mean and unit variance, preventing internal covariate shift during the large dimensionality reduction from 4,096 to 2,048.
Activation 1	reluLayer	,	'relu1'	Rectified Linear Unit activation. Introduces non-linearity enabling the network to learn complex speckle-to-DMD mappings. Chosen over sigmoid/tanh in hidden layers to avoid vanishing gradient during backpropagation

				[20]through the 8.4M-parameter first layer.
Regularisation 1	dropoutLayer	0.3	'drop1'	Randomly disables 30% of the 2,048 neurons during each training iteration. Prevents co-adaptation of neurons and reduces overfitting to the 8,000-sample training set. The 0.3 rate is higher than typical CNN rates because fully connected layers with 8.4M parameters are particularly prone to memorisation.
Hidden 2	fullyConnectedLayer	1024	'fc2'	Further compresses the 2,048-dimensional representation to 1,024 dimensions. This layer contains 2,048 × 1,024 + 1,024 = 2,098,176 trainable parameters. The progressive width reduction (4,096 → 2,048 → 1,024) creates an information bottleneck forcing the network to learn compact speckle representations.
Normalisation 2	batchNormalizationLayer	,	'bn2'	Second batch normalisation layer stabilising gradients before the final hidden activation.
Activation 2	reluLayer	,	'relu2'	Second ReLU activation for hierarchical feature extraction from the compressed representation.
Regularisation 2	dropoutLayer	0.2	'drop2'	Lighter regularisation (20% dropout) closer to the output. The reduced rate reflects that the 1,024-neuron layer has fewer parameters (2.1M vs 8.4M) and is less prone to overfitting.
Output	fullyConnectedLayer	1024	'fc_out'	Maps the 1,024-dimensional hidden representation to the 32×32 = 1,024 DMD output space. Each output neuron corresponds to one macropixel in the DMD grid. Contains 1,024 × 1,024 + 1,024 = 1,049,600 trainable parameters.
Probability mapping	sigmoidLayer	,	'sigmoid_out'	Applies the sigmoid function $\sigma(x) = 1/(1+e^{-x})$ to each of the 1,024 outputs, mapping them to [0,1]. Each value represents the predicted probability that the corresponding DMD

				macropixel should be ON. Binary reconstruction is obtained by thresholding at 0.5.

*Table 12- Forward **MLP** Model Architecture*

b) RL Model Implementation

i. MATLAB Code Development

Counter to the **MLP** model implementation, the **PPO** model moves away from the structure of supervised learning, and towards an Actor-Critic architecture, branching the learning method towards reinforcement learning by interaction with the environment, employing a structure, mirrored in the code implementation, exhibited earlier in Figure 9.

The architecture of the **PPO** model, detailed in Code Snippet 6 is modelled to learn to predict DMD pattern configurations by interacting with its simulated environment to offer reward signals derived from accuracy, traversing through trial-and-error phases to ultimately arrive at an educated understanding of the relationships between the speckled image and its ground truth, the target DMD pattern. To learn from reward signalling, the model must accommodate high-dimensional inputs. Table 13 outlines the full specification. The model initially handles the intractability problem that arises from a full resolution dimensionality of the full speckle capture (4096 pixels) by implementing a PCA compression step directly before model training. A fully connected layer without this format represents a parameter size not suited for CPU-constrained training, with the mitigation being handled in the **PPO** model through PCA compression. Concerning Behaviour Cloning (**BC**), the process before fine-tuning to fully initialise the policy, the actor network is pretrained using a 20-epoch supervised training run, providing weights to transfer to the main **PPO** network. The rationale of applying these components is to address the inability of a naive **PPO** in discovering useful policy gradients, as the 1024 dimensions that correspond to the individual macropixels cannot indicate which of the pixels should activate. The actor network and action space provide continuous control, with the **PPO** employing a separate network, the Actor-Critic, with the actor referring to the policy, and the critic representing the individual estimation, to stabilise learning, with the Actor being implemented with the rlContinuousGaussianActor function with two output heads. The critic network is a rlValueFunction, with an architecture of 256 → 1 to simply represent the state value. This architecture identifies the policy distribution, with the actor mean and std heads representing the Gaussian distribution that comprises the policy.

Component	Configuration	Rationale
Observation Space	PCA compression to 256 dims	4096-pixel input reduced to 256 components via compression
Action space rlNumericSpec	1,024 dimensions	Output passed through [0,1] sigmoid function for thresholding to determine reconstruct binary pattern decisions
Actor network trunk rlContinuousGaussianActor	256→ 512→ 256	Stabilise training without divergence
Actor mean head	256 → 1024	Set to avoid random initialisation collapse from BC
Actor std head	256 → 1024	Learns per-action standard deviation to encourage fine-tuning
Critic network rlValueFunction	256 →512 →128 →1	Used to calculate value estimation.

Behaviour cloning pre-training trainnet	20 epochs	Pre-training to initialise actor to performance and prevent cold start
PPO fine-tuning rlPPOAgent	30,000 episodes	Episodes are single step with 1 per episode, for full evaluation of the reconstruction policy.
Clipped surrogate objective	0.1	Limits policy change per update to prevent erratic updates

*Table 13- **PPO** model technical specification*

ii. PPO Hyperparameter Specification

The selection rationale for the hyperparameters are to provide a seamless trade-off to balance CPU stability with model efficiency, with equivalent attributes to the **MLP** models where applicable. The deviations in Table 14 arise from the inherent differences is models and model architectures, with the **PPO** employing an experience horizon of 512 steps instead of a mini batch, so that the agent collects 512 experiences before updating a policy every iteration. Employing a clip factor of 0.1 alongside the entropy bonus weight 0.02 is another deviation of hyperparameters, stemming from the need for a **PPO** to explore further, because the BC pre-training it has applied. A challenge of this implementation of selected attributes may come from the level of compression applied by the PCA to make the model computationally feasible. The main trade-off involving quality is speed of convergence, with the compression of inputs allowing for a rationale of information loss hindering model performance.

Hyperparameter	Attribute	Rationale
Optimiser	Adam	Same per-parameter learning as **MLP**
Learning Rate	$1x10^{-4}$	
Max Episodes	30,000	Sufficient value to assess convergience
Experience Horizon	512 steps	Fits memory for estimating policy returns
Mini-Batch Size	128	Equivalent to **MLP** implementation
Entropy Bonus Weight	0.02	Small value to reduce policy entropy collapse
Gradient Threshold	1.0	Clips gradient in early training

*Table 14- **PPO** Training Hyperparameters*

Parameter	Function	Value	Rationale
obs_info	rlNumericSpec	[256, 1] with LowerLimit = -inf, UpperLimit = inf	PCA-compressed 256-dimensional observation space; no clipping applied to the continuous components.
obs_info.Name	-	'speckle_pca'	Descriptive name identifying the PCA-compressed speckle observation.

act_info	rlNumericSpec	[1024, 1] with LowerLimit = -ones, UpperLimit = ones	1,024-dimensional continuous action space in [-1, 1]; output passed through sigmoid for [0,1] binary thresholding.
act_info.Name	-	'dmd_logits'	Descriptive name identifying the DMD control logits before sigmoid activation.

*Table 15- **PPO** Observation and Action Space Specification*

c) Machine Learning Implementation Sub-System Testing

To observe the suitability of a configured model prior to full-system actuation, individual components underwent an expensive sub-system testing scheme with a limited dataset to establish baseline performance metrics and verify the suitability of the full-scale implementation prior to final system testing.

To validate the working architecture, tests were conducted, with key practices involved alongside the model checked alongside both quantitative and qualitative criteria to pass validation. The goal would ultimately print the status of the individual 8 objectives, with examples found in Code Snippet 7, proving that the architecture is entirely capable of training. Interpreting the metrics reveals, with the quantitative validation objectives being assessed. The figures found in the sub-system test would require the model to diverge away from random figures, hinting that the model can extract features instead of memorising. The findings in Table 16 confirm these, considering that the MSE value extracted for the Reconstruction check is 0.243, a reduced figure from a random baseline MSE guess (0.375). The sub-system testing is incapable of generating the full solution, and it is only given 10 data pairs, the fact that the metrics indicate $r = 0.170$, and binary accuracy being 55.57%, show that the testing concludes that the mean correlation (r) of the predicted pattern and expected output is weak and the binary accuracy is above chance, showing that the models are learning, proving that both architectures are suitable to proceed to testing and validation using the 8000-sample dataset, to ultimately be systemically tested against the held-out 1000 sample testing dataset, as described in Final System Testing and Specification Validation.

Validation objective	Pass criterion	Rationale	Success?
Validity of architecture	Forward and inverse configurations loading successfully	Prevents runtime errors.	PASS
Training completion	Full training section completes without error passing	Shows model can compute through training in its entirety	PASS
Output shape conformity	Output shape matches [32×32, 1]	Matches 32x32 grid for hardware actuation	PASS
Output range validity	All predictions in range of [0,1] after sigmoid	Correct format with binary thresholding of 0.5	PASS
Reconstruction check	MSE diverges from random baseline	Concludes model can learn patterns	PASS
Classification accuracy	Accuracy function diverges from random baseline	Concludes model can learn classification	PASS

Spatial correlations	Pearson $r > 0$ between prediction and ground truth.	Shows model can correlate speckle to DMD patterns	PASS
Visualisation integrity	Prediction plots render without error loading in correct format	Enables quality verifying through training process	PASS

Table 16- Sub-System Validation Objectives and Results

3) Final System Testing and Specification Validation

3.1 Testing Setup

The testing setup establishes a reproduceable and equally rigorous framework of validating the model architectures against the deliverable goal, with the contrasting models equally assessed with comparison being explored in 3.4 Comparative Analysis. An imperative feature that is a key feature of the testing process is ensuring that final evaluation is strictly decoupled from the hardware framework and guarantee no biases arise from analysis. Every iteration of the testing paradigm undergoes five primary evaluations against a held-out 1000-sample sample set that define the diagnostic parameters that will be used to infer performance bounds that characterise both systems' behaviour. Seen in Table 17, the main quantitative and qualitative diagnostic metrics offer a direct comparison to specification targets, with merit detectors being further explored in section 1.3.3. The key figures that summarise Table 17, alongside the final system testing, isolates the hardware and data acquisition setup from the testing, which is done entirely post-training. This allows for performance deficiencies being able to be attributed to originate from model architectures and isolate issues in training or model hyperparameter selection as opposed to hardware artifacts, potentially diminishing the project performance against key deliverable selections.

MSE & Binary accuracy%	SSIM	Jaccard Index (IoU)	Confusion Matrix (TP/FP/FN/TN)	Threshold Sensitivity Sweep
0.02 < MSE < 1	-1 < SSIM < 1	0 < IoU < 1	$0 < F_1 < 1$	[0,1]

Table 17- Quantitative and qualitative diagnostic metrics for system validation

All final testing offers reproducibility, ensuring that the fixed random seed (rng(42)) is kept as a guarantor to ensure that the 10,000-pattern data split provides the same partition of training, validation and test data, seea) Data Acquisition. The significance provides that future testing data have not already been seen by the learned models, providing a biassed assessment of outcomes, detrimental to the deliverables of the project. The test speckle dataset is evaluated to characterise system behaviours along Table 17, which map directly with rubric goals **D3-D6**.

- Reconstruction metrics MSE and SSIM enable quantitative validations that directly measure the accuracy against the ground truth. This will allow the specification validity to foremost assess the visual proficiency.
- Classification diagnostics, such as the Jaccard Index (*IoU*) and the Confusion Matrix derive statistical analysis on DMD pattern mapping and the correlation they have with the predictions the model make, suggesting systematic imbalances across the individual model. The confusion matrix will correctly assess the systematic biases towards certain predictions, an imperative indication of the capability of the models' capabilities in learning unseen data, opposed to general speculation.
- Threshold Sensitivity assesses practical effectivity of the decision-making used for DMD pattern reconstruction, and how certain any given model is of their inference.

The testing setup involves an isolated environment, involving CPU-only conditions, in which computational efficiency will be easily evaluated, highlighting computational efficiency values compared amongst models, with the full hardware specifications of the test environment detailed in 2.2.1 Machine Learning Model Architecture.

3.2 Reinforcement Learning Performance Validation

Evaluation of the Proximal Policy Optimisation (**PPO**) reinforcement learning followed on from b) RL Model Implementation, specifying the mapping of PCA-compressed speckle patterns, observing through 64x64 dimension maps the binary prediction capability of how a reinforcement learning based model intelligently controls wavefront shaping through a trained actor, against a held-out 1000 sample training dataset.

3.2.1 Training Convergence Analysis

In a practical sense, the indicative assessment of the model's suitability of validating whether a training stage has a capacity of acquiring a stable and converged inference policy, is to assess the behaviour cloning warm-start phase over the 30000 episodes of training, with Figure 15 plotting the per-episode reward over the training stage, overlaid to detail the **PPO**'s convergence behaviour on creation of its policy.

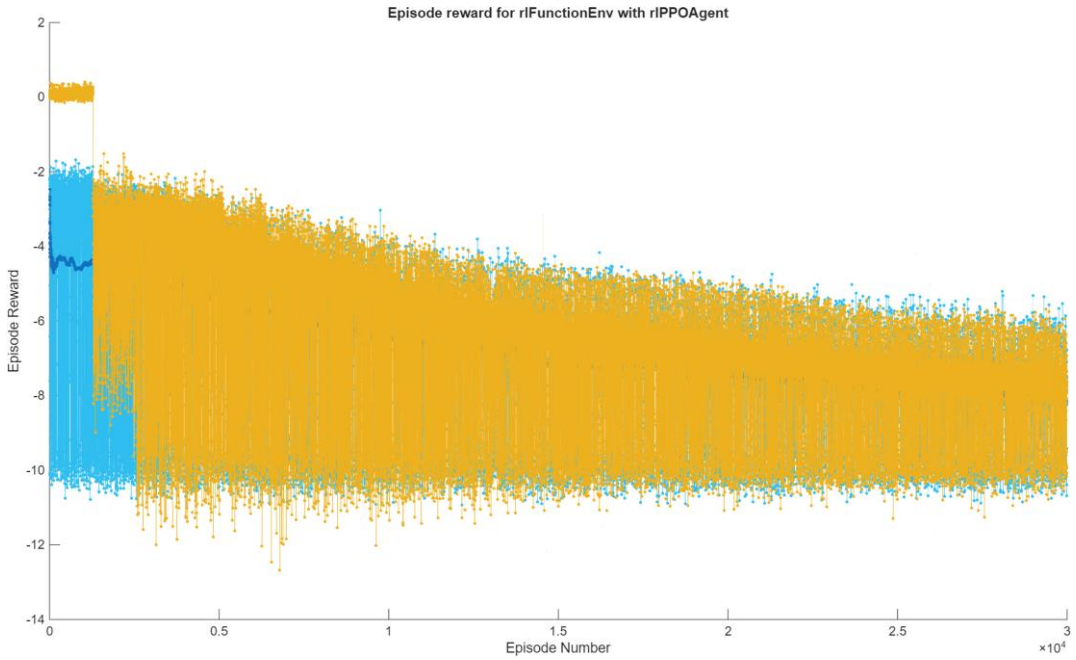


Figure 15- Episode reward trajectory via MATLAB Reinforcement Learning Training Monitor

The reward curve appears to exhibit a scattered fluctuation of values of reward allocation each episode, consistent with the stochastic behaviour of the fine tuning process it exhibits, rapidly inferring upon trial and error and allocating rewards based on the MSE loss that is calculated directly through the models prediction of Equation (9) (which is also noted in Figure 15):

$$Reward_t = -\tfrac{1}{5}\mathbf{MSE}+0.02\text{xHD}$$

where HD refers to the Hamming distance. Further analysis of Figure 15 confirms the nominal plateau in the first 3000 episodes ($0.3\text{x}10^4$in axis), which is proceeded by a near-zero average 0 value reward in the first 1000 episodes ($0.1\text{x}10^4$in axis), the explanation of this stems from the BC warm-start phase, with the **PPO** fine tuning module improving on baseline figures in the early episodes. The reward curve prior to this clipping function was provided a validation MSE of approximately 0.3 supplied from the behaviour-cloning module, meaning that roughly half of predictions were initially correct. The final measurement of mean reward corresponds to approximately zero, set to converge, with the training MSE figure remaining comparable to suggest the model initially achieving the primary goal of not overfitting to its training distribution. Another sanity examination of the training process was to provide proof that the training reached a plateau, with increased increments of episode rewards symbolising meaningful policy changes. Achieving a stable episode reward is the objective of having a model reach a final inference plateau, showing minimal improvements from further episodic cycles. Figure 15 demonstrates a nominal downward trend from episodes 3000 to 22,000, after which the gradient trend flattens, signifying the exhausting of marginal gains from further testing, proving the model has sufficiently reached a guided inference.

3.2.2 Aggregate Performance Metrics

Several findings, upon initial aggregate performance testing establish the sentiment overall that surmises the model performance. The values in Table 18 represent the evaluation of the **PPO** model from the test dataset, which is the aggregate performance, performed by the model before per-sample analysis realised in section 3.2.3. The purpose of Table 18 is to reveal the structural biases and baseline statistics that resulted from the testing of the model. Features that can be interpreted is the challenge the model faced regarding the 1000 sample dataset, with a strong binary accuracy value of 84.95% proving that significant classification performance was achieved by the model, superseding the naive baseline, and provides that, at a system level, that the **PPO** model is able to handle sufficient classification tasks. This can be duly accessed against the specification, as at a baseline point of view, it is seen in the testing dataset through coding achieved by Code

Snippet 8 that 42.25% of all DMD pixel configurations are ON, with the remainder configured as OFF. A naive classifier which remains static and predicts every pixel as off would achieve a binary accuracy of 57.75% and with the binary accuracy exceeding that value, the classification goal of the specification would be achieved, proving the model's competency against the specification.

Metric	Value
MSE	0.124173
Binary Accuracy	84.95%
SSIM (mean ± std)	0.5868 ± 0.4278
SSIM (5th /50th /95th percentile)	-0.0088 / 0.7079 / 1.0000
PSNR	Inf dB / 8.26 dB
Precision/Recall/F1	0.7801 / 0.8983 / 0.8350
Inference Latency	0.058 s
Jaccard Distribution	0.6219

*Table 18- **PPO** Aggregate test metrics*

Regarding model accuracy, the **PPO** implementation does not achieve consistency, as the standard deviation of the per-sample SSIM (Figure 16) is 73% of the mean, comparing the values in Table 17's SSIM (mean ± std). This would suggest that the model lacks consistency, in addition to the large discrepancy between the extreme percentiles. The model conflates the correct patterns with the incorrect, meaning that predictions are unreliable. Another observation from the SSIM distribution is realised in the indication seen in the 5th percentile negative SSIM value and at 5% of tests, the predictions are anti-correlated with the true pattern, with the extreme bimodality, the 95th percentile indicating the perfect reconstruction value of 1.00 being achieved 5% of the time. Evaluating this, the modal prediction performed by this model was entirely correct, however, the spread of the central 90% of the data spanning from a negative value to 1 would indicate the model having a bimodal performance against test data, where complete failure occurs occasionally. The Figure 16 histogram can be described from a bimodal distribution towards two modes, with the primary mode being centred at SSIM = 1.00, being achieved by approximately 36% of the 1000 test samples. Critically, the secondary mode is near zero, centred at SSIM = 0.00, with 19% of the test samples, representing that the second most common outcome concludes with the **PPO** model predicting a pattern with no structural similarity with the test pattern. This can also be interpreted that, in a better trained interpretive model, the test prediction would be observed as unimodal distribution, with its mean and median value being represented by a sizeable portion of the individual samples, contrary to Figure 16.

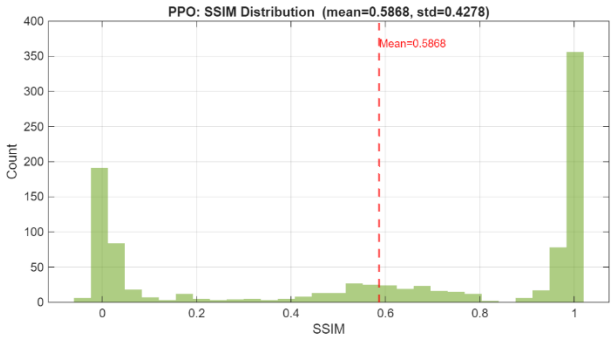


*Figure 16- SSIM per-sample distribution for **PPO** model*

The mean PSNR value in the aggregate test is reported as Inf dB, which upon inspection reveals that the MATLAB PSNR function returns infinity when the MSE is exactly zero, the second value in Figure 17a shows that the median PSNR is 8.26 dB, representing the median quality of the prediction samples, motivating the per-sample analysis. The more distributive per-sample findings can be contrasted with the aggregate findings for a richer interpretation, with an example being the PSNR. Of the 1000 test samples, 318 samples exhibit infinite PSNR, with the extreme bimodality of the testing outcomes is consistent with a 3 in 10 chance that the **PPO** model reconstructs a pattern to identical quality without pixel error entirely. The

remaining 68.2% of samples exhibit finite PSNR values with a median of 8.26 dB and in interquartile range of 3.15-15.02 dB, values extracted from Figure 17. Also, inspecting the spread of PSNR values indicates that its performance varies across the test dataset, suggesting that reconstruction quality is more dependent on the type of pattern rather than failing on chance. Figure 17b presents the overlayed plot comparing frequency of PSNR vs SSIM overlay. High quality pattern reconstructs cluster about the top-right quadrant, with 318 samples representing perfect or near perfect reconstruction quality. There is a low-quality cluster in the bottom left axis with SSIM < 0.2 AND PSNR < 10 dB, indicating poor spatial alignment. The visual separation seen in Figure 17 highlights the model's performance to specific training patterns and its limited learning of speckle formation.

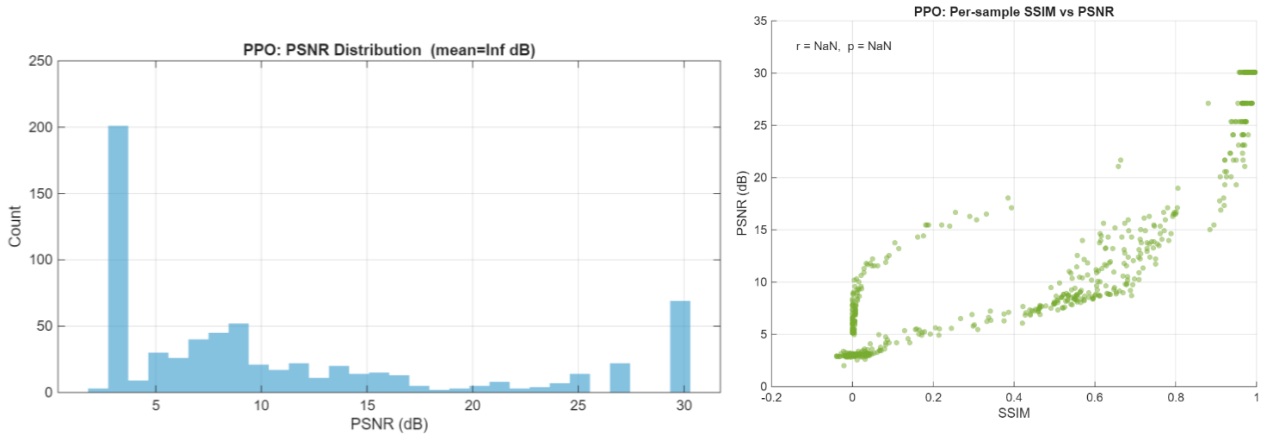


*Figure 17- **(a)** PSNR Distribution **(b)** SSIM vs PSNR comparison*

The other metric that analyses pixel-level classification that builds off the 84.95% binary accuracy is the behaviours the **PPO** model exhibits when classifying, away from the aggregate figure that combines all 1,024,000 (1000x 32x32 pixels) predictions into one figure. The confusion matrix figures assess the pixel-level decisions the model interpreted:

True Positives (TP)	False Positives (FP)	False Negatives (FN)	True Negatives (TN)
388,667	109,585	44,016	481,732

Table 19- Confusion matrix

From these values, Key classification metrics in Table 19 can be derived, with the model's classification of all pixels as ON being correctly identified 78.0% of the time, with the remaining 22.0% of the ON predictions being false alarms, leading to derive the Precision score of 0.7801. Of all ON pixels in the ground truth, 89.83% of them were correctly identified by the model, leading to a Recall score of 0.8983, also meaning that the F1 score, the harmonic mean of both the precision values and the recall, in being 0.8350 as a result. with the higher values of the Recall score, the model is biased to predicting ON pixels, and the value discrepancy between FP and FN proves that the model tends to produce a substantial amount of false ON predictions, when on aggregate 42.25% of pixels in the dataset are ON. This could explain the bimodal nature of SSIM distribution, as the excess of ON predictions degrading through the inference process. A more representative value that would account for class imbalance.

3.2.3 Qualitative Validation

Carrying on from the notion in section 3.2.2 regarding model quantitative metrics, Quantitatively, the selection of best vs worst performing pattern reconstructions, Figure 18, provides a best performing sample, Figure 18a, with a SSIM = 1.00, indistinguishable from the ground truth DMD patterns. It represents 1 of numerous perfectly reconstructed samples. this shows the model capacity for perfect reconstruction. The worse performing pattern represents the cases when the model prediction exhibits no visual resemblance to the target pattern, with output, representative in Figure 18b, resembling random binary noise, with no spatial structure representative in the dataset classification, meaning reconstruction has failed entirely. The nature of

the binary choices that the model is required to classify results in the worst-case failures exhibited in this model confirms that overall, the classification is prone to bimodality, with success in less sparse patterns, and abject failure in other deviating configurations. Figure 18a suggests that the model is likely to be more accurate the less degrees of freedom a dataset has, as it perfectly predicts 16 out of 16 nodes, compared to a pattern with 32x32 macropixels, not being able to distinguish 1024 configurable nodes.

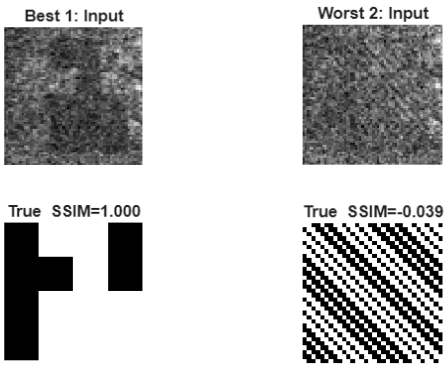


*Figure 18- Example of best **(a)** and worst **(b)** SSIM values exhibited by the **PPO** model with corresponding speckle-ground truth pairs*

A key measure is output characteristic analysis, which will assess the confidence of the model against the specification, probing the analysis of the model's output confidence, presenting the binary pixel accuracy as a function of threshold value. Figure 19 presents the accuracy against the threshold, this a model with more intermediary probabilities outputting a stronger threshold confidence, as opposed to a model that outputs values near 0 or 1, proving threshold insensitivity, and meaning that predictions are highly confident. Figure 19 assesses the binarisation threshold sweep results, with the median value of 85.0% at 0.5 representing the peak binary accuracy. The model exhibits threshold insensitivity, which would suggest that the model failure is determined by the correct DMD pattern samples output values near the decision boundary. However, inputs outside training produce outputs that fall near the decision boundary. The binarisation threshold rapidly increases to the median value at 0.2, signifying that the model passes the accuracy threshold against the specification, as it exceeds the 70% binary accuracy criterion.

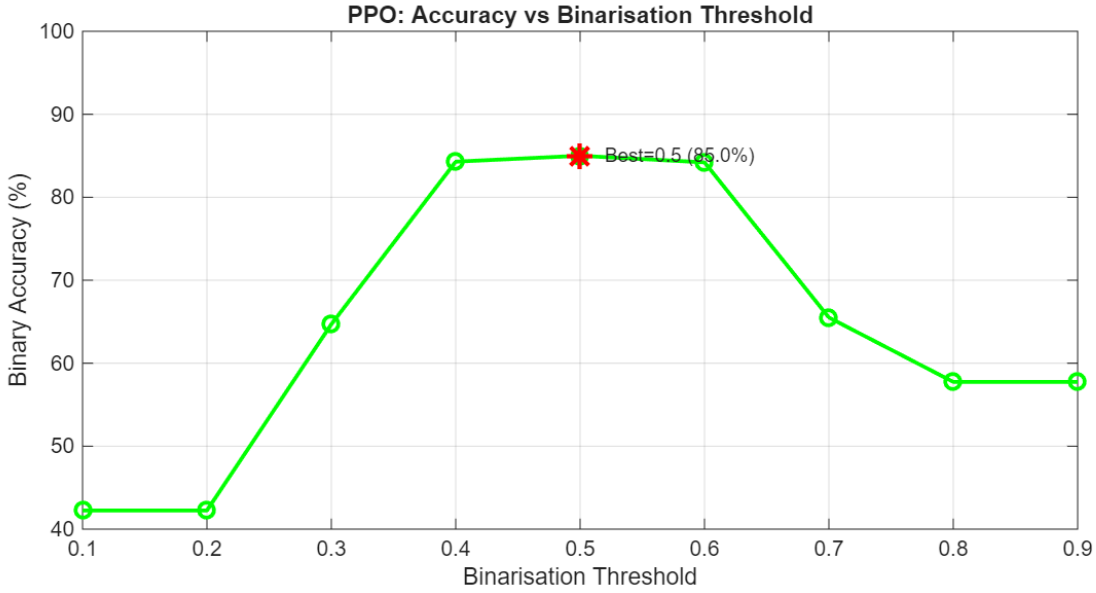


*Figure 19- Threshold Curve for **PPO** model*

3.2.4 Summary and Specification Assumptions

The **PPO** model, withheld from being trained on the 1000 testing pattern faced a varied test performance, with the comprehension of the aggregate and per-sample statistics pointing to, in layman's terms, the model capping at a certain pattern difficulty. The model exhibits perfect competency to a certain extent, with the bimodal nature of statistical findings resulting in inconsistent genuine learning capabilities for this specification. An overall indicator of this assessment is reflected in the Jaccard Index mean value (*IoU*) of 0.6219, exhibiting a mean value that is in the passing range, clearly visually displaying, in Figure 20, a distribution that mirrors the SSIM bimodality, with samples mirroring the perfect *IoU* and the near zero *IoU* values being on opposite ends, demonstrating the bimodal nature and ultimately the models aggregate figures representing the average findings across two distinct subpopulations.

Deliverable	Specification	Measurement	Result
D3/D4	Binary pixel accuracy ≥ 70%	84.95%	Pass
D3/D4	Mean SSIM ≥ 0.60	0.5868	Fail
D3/D4	Mean MSE ≤ 0.08	0.124	Fail
D5	Median PSNR ≥ 10 dB	8.26 dB	Fail
D5	Jaccard Index *IoU* ≥ 0.50	0.6219	Pass

*Table 20- **PPO** Specification Validation*

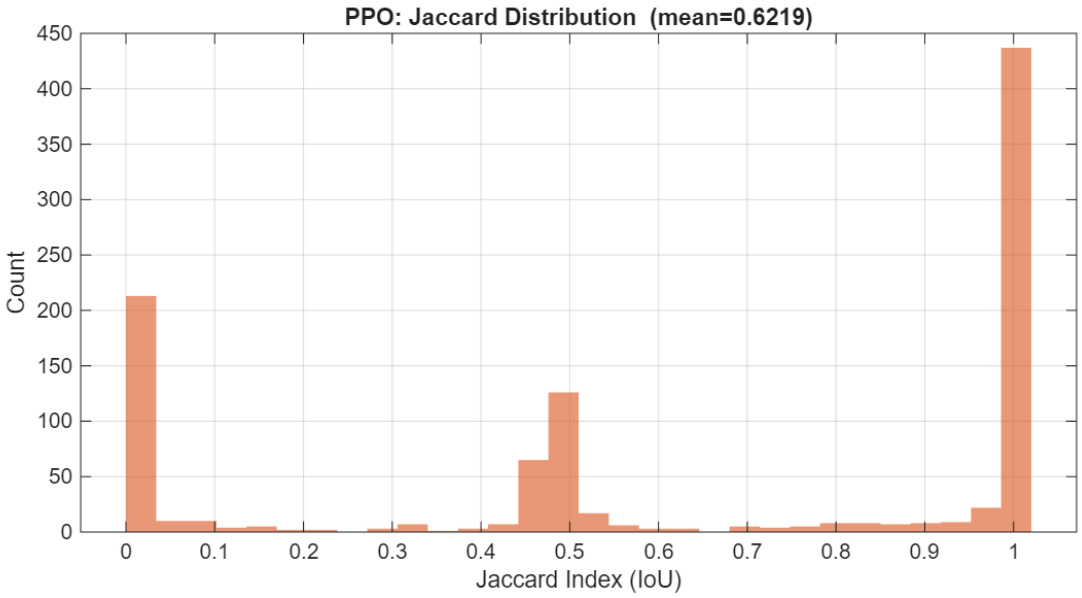


*Figure 20- **PPO** Jaccard distribution histogram*

The findings support the given conclusion against specification baseline assessment in the finding that the **PPO** agent is overall unsuited for mapping high-dimensional pattern mappings and can reconstruct patterns perfectly to a limited number of configurable nodes.

3.3 MLP Performance Validation

Evaluation of the Multilayer Perceptron (**MLP**) supervised learning followed on from section 2.2.4a, specifying the direct mapping of full-resolution speckle patterns to binary DMD configurations, observing through 64×64 dimension maps the binary prediction capability of how a supervised learning model reconstructs DMD masks through a trained feedforward network, against a held-out 1000 sample test dataset. Two architectural variants are evaluated: the Forward model and the Inverse Controller model, with this section addressing the Forward model performance.

3.3.1 Forward Model Validation

i. Training Convergence Analysis

The assessment of the model's suitability of training stage stability involves acquiring a the converged, is to assess the supervised learning phase over the full training duration, with Figure 21 plotting the training and validation loss curves over the training stage, detailing the **MLP**-Forward model's convergence behaviour on creation of a stable inference.

Analysis of Figure 21 confirms the nominal convergence trend in the first 400 iterations, which is proceeded by a plateau of the validation loss settling at an RMSE of approximately 8. The absence of divergence between training and validation loss indicates that overfitting to the training distribution is not severe, the model converges to a stable solution that generalises to held-out data to a measurable degree, in contrast to the **PPO** agent whose reward plateaued early and whose test performance revealed extreme bimodality.

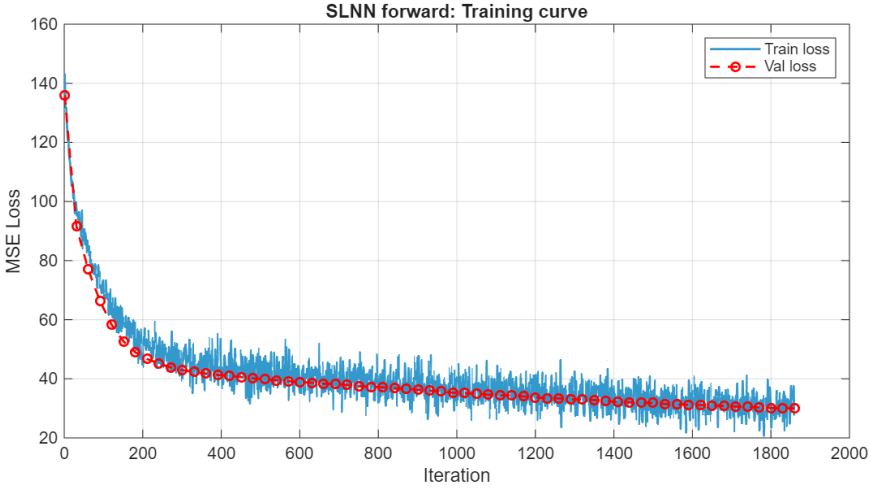


*Figure 21- Forward **MLP** Training and validation MSE loss over full training duration*

With training loss represented with the blue solid lines and the validation loss represented with the red circle markers, the loss curves are mostly overlapping of each other, both starting at 139 and dropping over the next 200 iterations and gradually converging to settle after 1600 iterations. The training loss exhibits a high-frequency oscillation; however, the validation loss continuously fits to converge, further, with the fact that convergence occurs at 1600 iterations, that the model is not overfitting.

ii. Aggregate Performance Metrics

Aggregate performance testing for the forward model provides values in Table 20, that represent the evaluation of the **MLP** forward model. Table 21 summarises the baseline performance characteristics that resulted from the testing of the model, with all metrics exceeding specification. Features that can be interpreted is the classification challenge the model faced regarding the 1000 sample dataset, with a held-out binary accuracy value of 90.33% proving that significant pattern discrimination was achieved by the model, and provides that, at a system-level, that the **MLP**-Forward model is able to handle sufficient classification tasks. The classification goal of the specification is met, proving the model's competency against the naive baseline. The **MLP** forward model features a comfortably exceeding binary accuracy compared to the specification, with it meaning that the model generalises well to unseen data.

Metric	**Value**
MSE	0.061248
Binary Accuracy	90.33%
SSIM (mean ± std)	0.7081 ± 0.3746
SSIM (5th /50th /95th percentile)	0.0064 / 0.8834 / 1.0000
PSNR	Inf dB / 9.50 dB

Precision/Recall/F1	0.8893 / 0.8809 / 0.8851
Inference Latency	0.55 s
Jaccard Distribution	0.7436

*Table 21- **MLP** Forward Model Aggregate Test Metrics*

Additionally, the per-sample metrics seen in Figure 22 present an overlaid contrast of findings alongside the inverse metrics, detailed in section 3.3.2. The data is presented in this way as they were deemed to be comparable, and the distributions of the models would be seen to overlap with each other. Throughout the combined figures, the forward model is presented as blue, the inverse in red. Regarding SSIM distribution of the forward model, Figure 22 exhibits a spike at SSIM = 0, where approximately 70 samples are attributed to having this low value, representing around 7% of all test cases. Across the interquartile range, the blue histogram is relatively evenly distributed, with the larger (perfectly reconstructed) spike occurring where SSIM = 1. The forward model exhibits a spread across intermediate values and has a somewhat substantial number of intermediate values at the non-polar ranges. Figure 22 still suggests that 46% of samples have a perfect reconstruction, and that the model is likely to perform well on average.

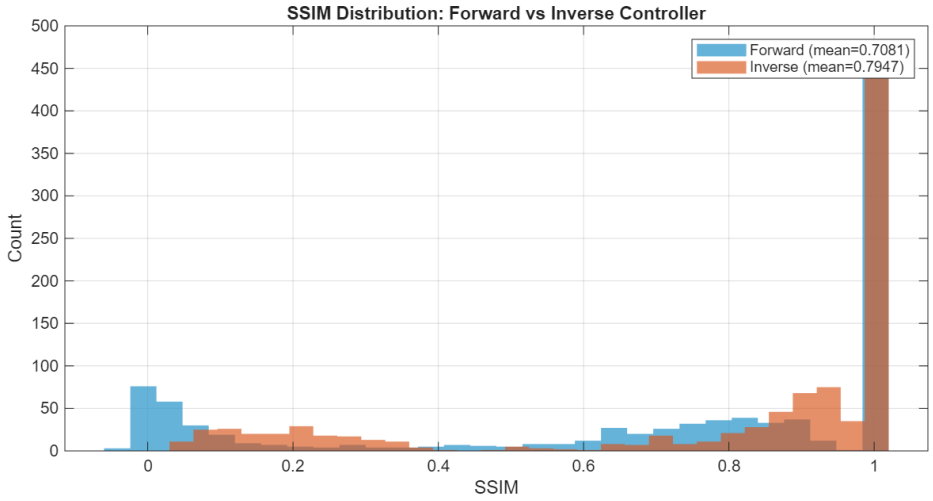


*Figure 22- **MLP** Forward and Inverse Controller Performance Distributions*

Regarding the PSNR values of the forward model, a broad distribution of values, seen in Figure 23a, has a very low PSNR cluster concentrated at PSNR < 4 values, indicating the formation of worse reconstruction, with the second cluster that it has in the range of 15-20 dB indicating the ranged and varied sample presence in intermediate quality. The findings demonstrate the full range of classification the model achieves, without having extreme failure cases, separating perfect and partially reconstructed data in testing. The reason that both models have combined SSIM vs PSNR findings is to highlight cross-model differences, with both findings in Figure 23b showing the non-linear correlation the SSIM has vs PSNR in both types of system, at SSIM = 0, the PSNR is capable of reaching 10 dB, showing that there is not a strictly linear relationship between the two values, hinting that **MLP** classification has structural nuance beyond simple accuracy. The **MLP** models differ regarding the differences between SSIM and PSNR, meaning that each metric captures distinct aspects of reconstruction quality.

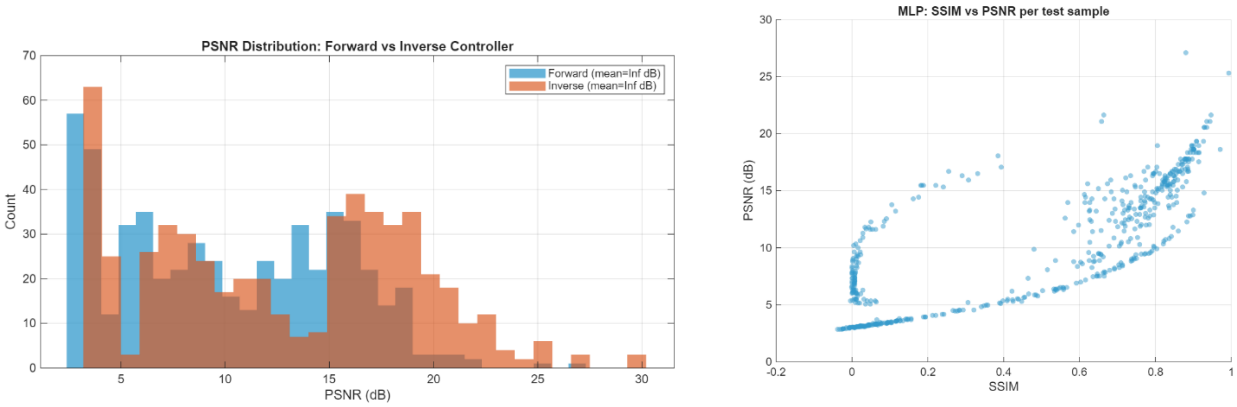


*Figure 23- **(a)** PSNR Distribution **(b)** Combined SSIM vs PSNR for **MLP** models*

The forward model's mean SSIM value is 0.7081, a moderate value that, coupled with the standard deviation value of 0.3746, hints at the per-sample variability of the predictions. With the standard deviation being half of the mean, significant variability exists across samples. Figure 24 highlights the *IoU* distribution of the forward model, with the blue histogram showing the spike-then-spread pattern, highlighting the $IoU = 0$ prevalence in the forward model's testing outcomes, a significant failure mode. The intermediate values, with a mean *IoU* of 0.7436 reflects the model's moderate overall success. These could indicate the inherently bimodal capabilities of the forward model, as 45.9% of samples are perfect pattern reconstructions, whereas a smaller subset of data would catastrophically fail in pattern inference.

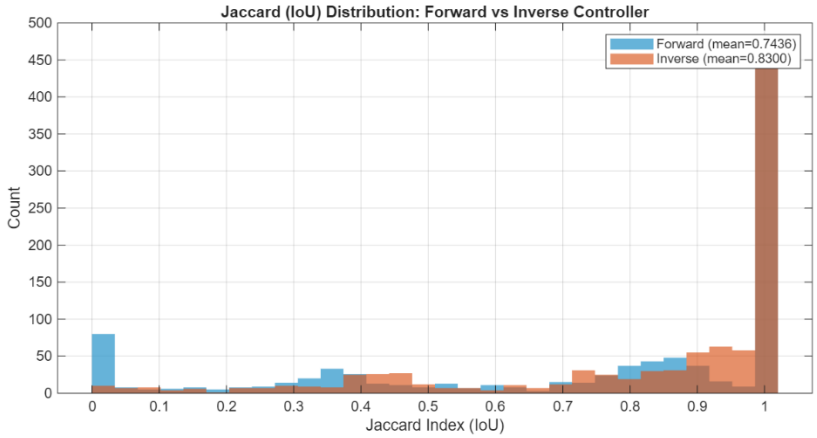


*Figure 24- **MLP** Forward Model IoU Distribution*

3.3.2 MLP Forward vs Inverse Controller Models Comparison

Table 22 presents the confusion matrices for both controllers against all pixel control nodes in the entirety of the test dataset. Unequivocally, the data shows the inverse controller outperforms the forward model in every classification metric. One point of reference is the precision values, where the Inverse controller scores on 91.56%, having a higher classification success of labelling ON nodes that genuinely were ON. The superiority of the inverse controller, with 10,365 fewer false positives, seen as a 21.8% reduction, which is taken to mean that the Inverse model memorised the DMD modes at a more efficient rate, with a cleaner on average pattern. Furthermore, when both models output an ON pixel, it is more likely that the Inverse model does so with a greater certainty.

Metric	MLP Forward	Comparison	MLP Inverse
True Positives (TP)	381,156	<	402,246
False Positives (FP)	47,466	<	37,101
False Negatives (FN)	51,527	<	30,437

True Negatives (TN)	543,851	<	554,216
Precision	0.8893	<	0.9156
Recall	0.8809	<	0.9297
F1 Score	0.8851	<	0.9226

*Table 22- Confusion Matrix Comparison: **MLP** Forward vs Inverse Controller*

As there is a total and significant discrepancy between the two models, the Inverse model also assumes a more correct identification of the true OFF pixels, with a sizeable difference in False Negative values, demonstrating that its superiority is balanced through its achievement of both a higher recall and precision score, with a trade-off in precision resulting in a prediction style that is more spatially accurate. This comparatively trades reconstruction breadth, and the physical basis of the difference arriving at the linear intensity assumption. There is also a beyond 0.9 value given in the inverse model's F1 score, a figure that shows the classification balance. The F1 score of 0.9226 indicates the improvement of balancing precision and recall.

The metrics in section 3.3.1 confirm the two models sharing similarities with distribution spreads and histogram shaping systematically appearing similar, however there is a glaring difference in all metrics, with the most notable difference being seen in the difference in cluster locations from the Jaccard Index diagrams, seen in Figure 24. The difference in comparison is the absence of $IoU = 0$ clustering from the (red) inverse histogram, which symbolises the absence of catastrophic failures. Although there is an intermediatory presence, similar with the forward **MLP** model, there lacks the near-zero cluster, which confirms the superior spatial fidelity. Another key difference is realised in Figure 23 where the PSNR distributions, with the inverse model overlaid in red, moves away from the relatively lower PSNR cluster values, with the inverse model histogram resembling a rightward shift of PSNR values, concentrating mass at a higher PSNR mean at a near identical distribution shape. Both controllers represent a distribution that doesn't tend towards any bimodal preference values, and their intermediate ranges are distributed broadly, which in terms of testing correlates to their consistent reconstruction. To structurally interpret the SSIM distribution from Figure 22, the Inverse model has its lowest density cluster at SSIM ≈ 0.2 value, a positive shift towards a coherent reading from the model, meaning inference at the lowest performance threshold results in a partially reconstructed pattern in that scenario.

i. Qualitative analysis of MLP models

The threshold sensitivity analysis in Figure 25 represents the threshold curves, with the similarity of operational perspective being reflected in the models both sharing the bell-shaped appearance that doesn't peak far off the lowest threshold value. The inverse controller is slightly at a higher performance, maintaining a 2-3% margin between the lower, forward model binary accuracy at each threshold level. This marginal advantage realises itself in the 93% peak value at the binarisation threshold of 0.5, approximating that the inverse model degrades in accuracy much less severely. The seemingly flat nature of the curves indicate both **MLP** model's strong confidence calibration, with the inverse controller outputting a clear indication of the superiority that the other metrics differed as an offset shift.

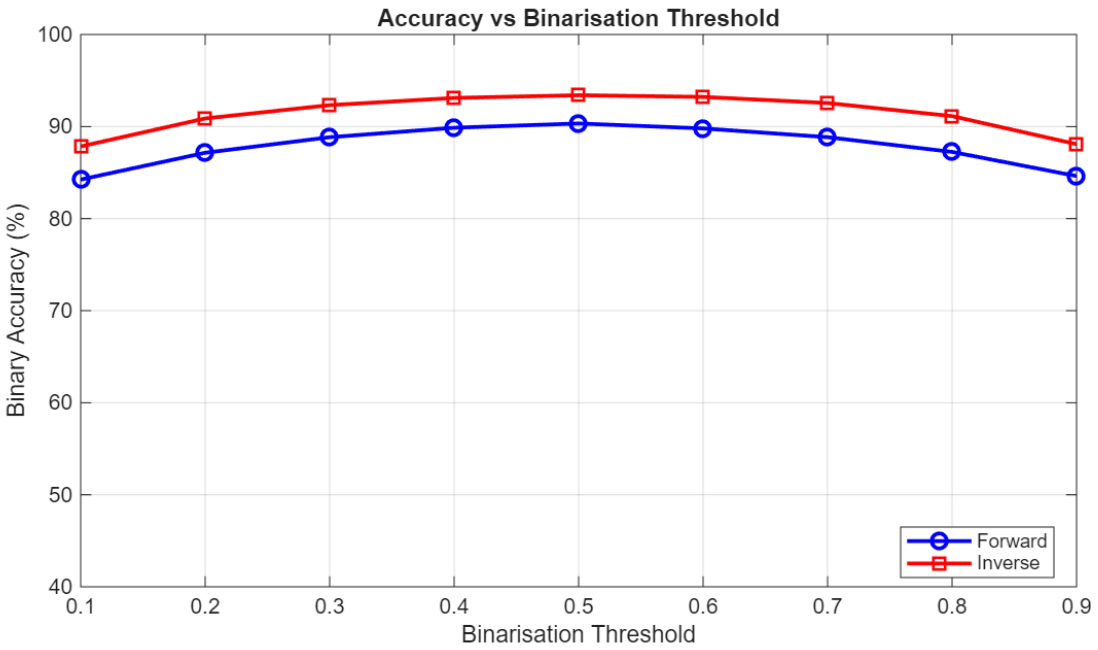


*Figure 25- **MLP** Forward and Inverse Controller Threshold Sensitivity Curves*

ii. The Inverse Controller

Systematically, the Inverse Controller is superior throughout the tested metrics incorporated in the validation testing, however, to examine the model on an isolated basis, the Table 23 values infer some details about the performance. Achieving a binary pixel accuracy against the held-out test dataset of 93.40% exceeds the specification threshold to a sizeable margin, with it indicating strong generalisation.

Metric	Value
MSE	0.044857
Binary Accuracy	93.40%
SSIM (mean ± std)	0.7947 ± 0.3126
SSIM (5th / 50th / 95th percentile)	0.1126 / 0.9550 / 1.0000
PSNR	Inf dB / ~15 dB
Precision / Recall / F1	0.9156 / 0.9297 / 0.9226
Inference Latency	0.47 s
Jaccard Distribution	0.8300

*Table 23- **MLP** Inverse Controller Aggregate Test Metrics*

Another revelation, also realised in Figure 26, arises from the 50th percentile sample exhibits a SSIM of 0.9550, which means, the median sample prediction is almost completely identically reconstructed. The per-sample analysis This confirms that the inverse controller only has around 3% of values at a sub 0.05 SSIM value. Conversely, around 460 of the 1000 test samples are perfectly reconstructed, meaning that the model reconstructs flawlessly 46% of the time, with SSIM = 1 being the most dominant figure, with a more prominent subset appearing with 0.7-0.9 SSIM, emphasising the structural fidelity of the inverse model.

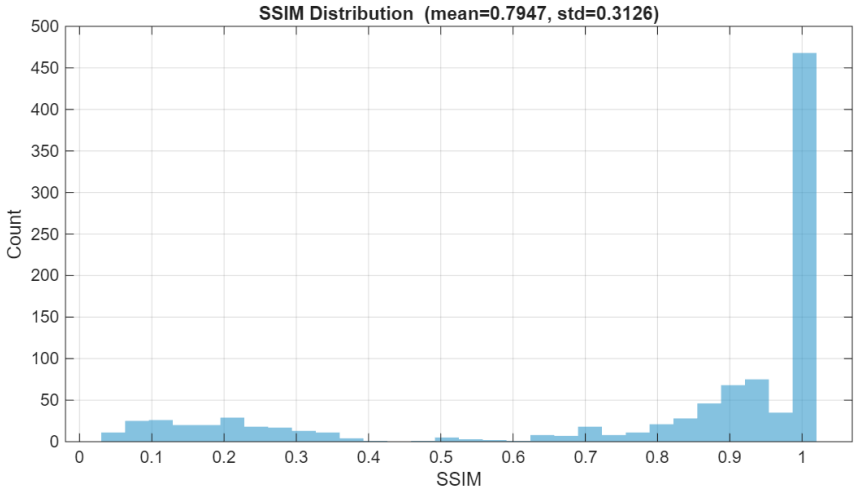


*Figure 26- **MLP** Inverse Controller SSIM Distribution*

3.3.3 Summary and Specification Assumptions

By being tested with the same 1000 sample test set as the **PPO** model, the models tested on the same specification provide imperative figures in their validation functionality. Regarding the forward model, the superseding of specification thresholds is highlighted, with binary accuracy achieving a much higher than specified value, otherwise, the specifications that also resulted in a pass are relatively marginally close to the specified criteria boundaries. This leads to evaluate that the forward model fully reaches deliverable threshold at **D3** and **D4**, indicating that although the model provides a baseline classification quality, it is unsuited at a larger scale, with the PSNR and *IoU* values highlighting a model inefficiency regarding complex pattern reconstruction.

Deliverable	Specification	Measurement	Result
D3/D4	Binary pixel accuracy ≥ 70%	90.33%	Pass
D3/D4	Mean SSIM ≥ 0.60	0.7081	Pass
D3/D4	Mean MSE ≤ 0.08	0.0612	Pass
D5	Median PSNR ≥ 10 dB	9.50 dB	Fail
D5	Jaccard Index $IoU \geq 0.50$	0.7436	Pass

*Table 24- **MLP** Forward Model Specification Validation*

Conversely, the inverse model passes the deliverables at **D3** and **D4** to a significant margin, demonstrating clear attenuation to high-dimensional patterns, and provides tens of percentage points worth of improvement towards the specification. Also, there is a unique quality exhibited by the inverse controller that also pushes it past the **D5** specification threshold, with a 5 dB margin in terms of median PSNR, allowing for a higher-quality benchmark. The findings support the conclusion of the model that it is more broadly operationally viable, through its **D5** specification pass confirmation.

Deliverable	Specification	Measurement	Result
D3/D4	Binary pixel accuracy ≥ 70%	93.40%	Pass
D3/D4	Mean SSIM ≥ 0.60	0.7947	Pass
D3/D4	Mean MSE ≤ 0.08	0.0449	Pass
D5	Median PSNR ≥ 10 dB	15 dB	Pass
D5	Jaccard Index $IoU \geq 0.50$	0.8300	Pass

*Table 25- **MLP** Inverse Controller Specification Validation*

Overall, the **MLP** models demonstrated broadly similar behaviours, with metrics compared between the two providing evidence that the inverse model exhibits more optimal performance in every metric. Due to lower percentile sample reconstructions, there were still errors that both models shared consistent characteristics with.

3.4 Comparative Analysis

Metric		PPO	MLP Forward	MLP Inverse Controller
MSE	↓	0.124173	0.061248	0.044857
Binary Accuracy	↑	84.95%	90.33%	93.40%
SSIM mean	↑	0.5868	0.7081	0.7947
SSIM std	↓	0.4278	0.3746	0.3126
SSIM 5th percentile	↑	-0.0088	0.0064	0.1126
SSIM 50th percentile	↑	0.7079	0.8834	0.9550
SSIM 95th percentile	↑	1.0000	1.0000	1.0000
Median PSNR	↑	8.26 dB	9.50 dB	15 dB
Precision	↑	0.7801	0.8893	0.9156
Recall	↑	0.8983	0.8809	0.9297
F1 Score	↑	0.8350	0.8851	0.9226
Jaccard Index *IoU*	↑	0.6219	0.7436	0.8300
Inference latency	↓	0.058 s	0.55 s	0.47 s

Table 26- Comparative metrics table where ↑ indicates a higher value is preferred and ↓ denotes a lower value is preferred

The **MLP** inverse controller is the strongest quality wise in its performance through testing and wins head-to-head against the project's other two models, with Table 26 accurately assessing the discrepancy between the inverse model and the other models. In classification quality, the linear progression of the models' improvements between the last one, seen in the pixel accuracy seeing a 5.38 and 3.07 increase, suggesting the choice of training paradigm, **MLP**'s supervision learning vs a reinforcement learning strategy, fares better in terms of basic classification for the former. The impact of model choice provides a meaningful assessment of classification capability, demonstrated in the SSIM and Binary accuracy values denoted by Table 26.

The F1 scores, that consolidate precision and recall, reinforce the interpretation of the imbalance of classification capabilities through the recall value of the reinforcement learning model being high contrasting with its acquired low precision, indicating the classification characteristics biasing towards an ON mode, despite the risk (and occurrence) of false positive generation, leading to an almost 10% margin in F1 score values.

The **PPO** model also adheres to interesting results by comparison regarding the 5th percentile SSIM values, which denote a negative (-0.0088) value, indicating, at the 5th percentile, that its worst predictions are anti-correlated, as opposed to both the **MLP** models, with the more precise of the two exhibiting a 0.1126 value, at its worst prediction, demonstrating that the Reinforcement Learning implementation is incapable of classifying at worst-case scenarios. Supervised learning has measurable structural fidelity that would still be able to be quantified at any point of the dataset, which the **PPO**, although it had the same dataset partitions, was not able to attain.

On pixel level error analysis, the Inverse Controller reveals that even partially failed predictions would attain a value of similarity to the ground truth DMD pattern, that is significantly higher than the 8.26 dB and 9.50 dB values seen in the **PPO** and Forward model, and combined with the Jaccard Index value of 0.83, provides the thought that the inverse model is significantly better at reconstruction than the other models, with the Jaccard Index predicting that the spatial overlap seen by the prediction is substantial.

With the results in Table 26, it is apparent the supremacy of the inverse controller implementation, however, the models face a trade-off in inference latency, a value obtained in the MATLAB environment that records inference time, the trade-off is significant in the Supervised learning paradigms, and result in the **PPO** realising approximately an 85% speed advantage across sample predictions.

3.5 Error and Limitation Analysis

The hierarchal structure realised in section 3.4 established the quality hierarchy, however, to some extent, all models share systematic failure characteristics that hinder their pattern reconstruction performance against the specification and stetch goal deliverables. All three models are affected by transcendent tangible properties that affect their performance, with the catastrophic failures in lower percentile generations appear to be affected by several non-learning factors.

The optical system characterised in section 2.2.2.3 employs a single scattering diffuser with a measured speckle contrast of $\mathbf{K}_s = 0.5046$ under all ON DMD illumination. This value, which represents almost half of the theoretical maximum $\mathbf{K}_s$ of 1 [1] is a resultantly difficult speckle pattern to assess determine binary classification through, exemplified by the inability to acquire a less dense diffusive lens to assess the Machine Learning based intelligent control. Another artifact of the physical diffuser stems from the non-uniform characteristics that were seen to impact model performance, with different speckle locations, which relate to spots of poorer and greater focus, providing marginally worse results with the models.

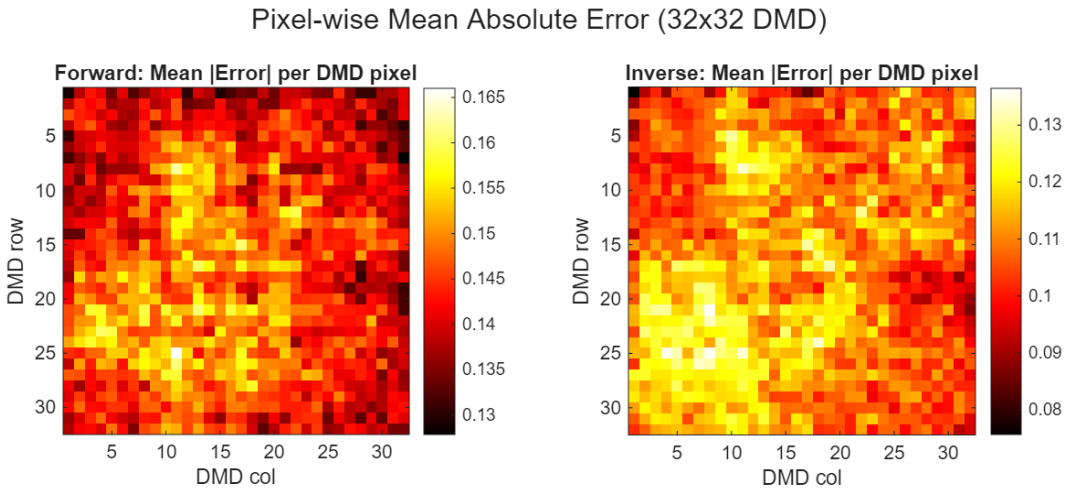


Figure 27- Pixel-wise absolute error seen by Supervised Learning

Figure 27 shows each of the 1024 configurable modes and their correlation with mean error that the **MLP** models would fare when predicting that individual pixel's binary configuration. The physical theoretically optimal value would result in an error map that mostly uniform, however there are different degrees of difficulty depending on what node is being predicted by any model.

A goal of this project was to account for computational load, and to optimise the Intelligent Control be efficient, however, a major roadblock that affected the testing was the hardware constraints, with dedicated GPU processing speed allowing for tens of times processing power, and would be able to achieve a higher amount of trainable parameters without affecting processing capabilities.

The **PPO** agent's defining computational compromise was the PCA compression, see section 2.2.4b, that reduced the dimensions from 4096 to 256, a 16x reduction that was necessary to allow for tractability of the model, trading a fraction of the speckle variance through the compression, with any reward function applied after compounding said information loss, which can attribute to the bimodality of the inference results.

4) Conclusion

4.1 Reflection on the Result

4.1.1 Project State Review

Core Deliverable	Completion	P/F
D1. Replication and calibration of experimental hardware	The deliverable was completed without issue, presented in 2.1.1 Experimental Setup and 2.2.2 Experimental Equipment Calibration, where the optical bench was optimised to include a baffle ring and calibration of the camera was tested in subsystems in c) Machine Learning Implementation Sub-System Testing.	PASS
D2. Automated image acquisition pipeline with managed preprocessing	The deliverable was successfully completed, with a full 10,000 pattern dataset, see 2.2.3 Dataset Development, correctly paired and aligned across a diverse dataset was realised, with preprocessing being attained in b) Data Preprocessing.	PASS
D3. Shallow **MLP** for speckle pattern processing	The deliverable was successfully achieved through the implementation of two architectural variants of the **MLP** paradigm, see section 2.2.4a and section 2.2.4b. Both trained with similar specifications to achieve fully testable models.	PASS
D4. PPO Reinforcement Learning agent for speckle control	The deliverable was achieved through MATLAB's reinforcement learning toolbox, using PCA compression for tractability, utilising a similar epoch period as **D3** Supervised Learning Models.	PASS
D5. Comparative Analysis of ML approaches	This deliverable has been completed successfully through the 3.4 Comparative Analysis comparison, with a structured analysis of all models and conclusions drawn in 3.5 Error and Limitation Analysis.	PASS
D6. Enhance dataset with varied experimental geometries	Included in a) Data Acquisition, the dataset has 9 stratified classes, with more complex data patterns achieved and included in the final dataset.	PASS
D7. Higher-resolution scaling and model optimisation	Addressed with the inclusion of **D6**, as the geometric patterns utilised a 32x32 grid resolution, moving away from 4x4 macropixel grid patterns, scaling the DMD to 1024 configurable nodes from 16	PASS
D8. Optimise to minimise computational load and maximise processing speeds	Partially achieved but not explored, system is completely optimised for the hardware specification that the project employs, however, there is no optimisation or balance of Model parameters, and maximisation of any variables attaining to that.	FAIL

Table 27- Deliverable Completion Summary

All five core deliverables had functionality successfully implemented, with the most crucial deliverables of implementing a functional model being established and built on by **D5** in model comparison. The paradigms produced results that could be analysed, with assessed suitability allowing for the model mapping capabilities in classification to be evaluated. It is later demonstrated that the established analysis resulted in a clear performance hierarchy, with the Inverse Controller architecture conclusively being the strongest performer. However, assessing suitability pertaining to the stretch goals provided worse results for **D5**, with the Machine Learning models somewhat struggling to attain respectable figures in accommodation of **D6** and **D7**. The nine classes that were included to satisfy **D6** allowed for a robust dataset, however, results with the **PPO** model resulted in a more bimodal classification behaviour, resulting in extreme and wayward errors faced in the bimodal subpopulation. **D8** was not fully implemented, as the control systems, apart from the inverse controller, were not concretely good enough to justify further decreases on computational load, with its trade-off being model performance.

4.2 Consideration of System Within a Larger Context

The intelligent control objective realised in this project is a proof-of-concept constrained by modest computational resources that would almost certainly never be used in commercial contexts, however, its strengths lie in the broader engineering practices that were displayed throughout the duration of the development and realisation. In context of further non-specified applications, there will be numerous dimensions that will have meaningful intersection, with the advancement of Machine Learning practices and imaging technology being fully relevant to broader optical control.

All experimental work conducted was housed in the Wolfson Building, with an optical workspace that required an approved risk assessment to be submitted regarding laser hardware that was used in the experiment. The VLM-520-03 LPT laser is classified as Class 1 under IEC 60825, indicating its safety for normal usage purposes, subject to nuance as operating with a standard safety procedures would be the professional standard regarding project safety. All optical equipment in Figure 3 was fixed to the workbench to ensure beam alignment to against gravitational or manmade misaligning movement, hindering the project almost entirely. The laser was only activated while acquiring data, and where the laser employed in this specific project holds no direct harm to operators, the practice of safe laser handling is scalable to laser handling projects in other fields, conducting risk-mitigation measures to protect users.

Principally, the project demonstrates that a supervised learning solution is the most effective solution for imaging-based wavefront shaping involving a DMD, with the implementation of my methodology emphasising the consumer grade hardware and accessible toolsets that was employed. The model, trained on CPU-only environment, uses MATLAB, and native MATLAB add-ons to assess the problem of a wavefront shaping control system, whether a potential replicant would upgrade the hardware's computational capacity, there is an indictment that my methodology is wholly replicable and that it is possible to be specialised towards any field that would characterise wavefront shaping research. No phase-only SLM, interferometric detectors, or dedicated GPU's [12]were used, commonly applicable hardware specifications of contemporary speckle propagation research. For laboratories or teaching institutions without access to any of those resources, or would want to achieve foreknowledge without expenditure, this project provides the guidelines, and model architecture to assess a low barrier, demonstratable solution to wavefront shaping.

Another by-product of this refers to the hardware used for the simulation environment, see section 2.1.1c, and the environmental implications, where, although not quantifies directly, is minimal and an insignificant cost. Deep Learning in a contemporary sense involves research employing state-of-the-art GPU accelerators, with thermal design power of up to 450 W [32]. Minimally the project and computational mitigations portrayed in the project has negligible power increases from standard use of the device, with a magnitude lower power consumption, allowing this project to not even quantify the direct energy consumption statistics. In a research landscape, the use of a shallow **MLP** architecture represents a responsible choice, with a baseline that doesn't consider the energy use implications analysed in this project.

For commercial standardisation, MATLAB, alongside MATLAB's Deep Learning and other ML based toolboxes used in this project, provide, alongside the use of the Texas Instruments DLP4710 DMD, a commercially deployable baseline for both digital modulation and processing of data and control of digital modulation, which, obviously, can be scaled to include other industrial expansions, such as 3D printing applications, or inclusion of other peripherals, with the modular approach of my methodology allowing for the separation and modification of the data acquisition, preprocessing and training for further research.

In the immediate term, it could be advised to have a single impactful upgrade to scale the project, with a more advanced CMOS sensor capable of a magnitude greater frame capture, to ensure that data acquisition can be made efficient, a challenge faced otherwise, with data acquisition times for 10,000 frames of dataset pairs easily being upgraded to be near instantaneous, this would also be paired with a range of different sensitivity diffusive lenses for a more robust and cost-friendly analysis. In a medium term, with a budget of £100-£500, a meaningful upgrade to streamline efficiency would be to acquire a Cloud GPU rental, to allow for more powerful models to be employed to assess scalability. With the longer term aspirations, a societally

beneficial application of this project would be to construct the pathway to allow for use in medical imaging, with optical noise present in tomography and laser endoscopy to be mitigated post-capture, with medical efficiency being a positive externality to society, and allowing for the natural scattering present in that field to be intelligently controlled by a supervised learning paradigm.

4.3 Reflection on Management

The emphasis on the project management was consistent, with a structural and weekly supervisor meeting, alongside PowerPoint slide presentations on weekly findings, tracked the progress of my project, alongside the support of the monthly proformas, evaluating strategies undertaken in the project scope. Additionally, this section would emphasise the efficacy of strategies that were beneficial for future engineering contexts. Through the duration of the project, beginning at the conception of my project proposal, an initial Gantt chart was listed to detail the proposed roadmap that the data acquisition through the wavefront shaping solution was going to undertake. It was imperative that in my initial pipeline of the flow of goals that were to be achieved, I would need to follow a sequential schedule using the Gantt chart, which chronicled my initial roadmap, which quickly deviated due to unseen variables arising from hardware complications. Components absent from prior project adaptation slowed down the projection of the workflow, with recalibration of the optical setup having to be redone, and re-characterisation of certain new swapped out hardware parts, with individual new calibration (**D1**) adjustments, which were not originally accounted for having to be worked on before the Gantt chart could resume its natural path. The new tasks, such as exposure settings, gain and physical positioning pertaining to the camera being implemented as an unseen necessity. These tasks prior were not on the original Gantt chart, with deviations of my methodology leading to the whole Gantt chart quickly becoming obsolete. The most significant deviation from the devised workflow arose from the addition of the scattering diffuser, a component that was absent from Mr. Jialu Zhu's project [28]and required recalibration of the camera optics, and the hardware setup, which ultimately lead to the setup employing a different Neutral Density Filter. Thes tasks were not foreseen, and the anticipated Gantt chart led to explicit displacements in timelines.

The most effective management practice which was employed consistently was the weekly preparation slides, alongside a 30-minute meeting with my supervisor, who had information to be relayed to me, which lead to informed decision making throughout the project implementation timeline. The slides, and mental meeting recollection, created a cumulative record that streamlined the decisions that were undertaken in the project, along with the thesis-writing process, using accounts to detail correct and incorrect decisions. The week-by-week structure allowed for practice and experimental periods of project implementation to culminate in feedback that strengthened my project's process. The roadblocks encountered were not immediately fixed, however with documented and effective feedback, whenever issues arose, there was a practical and tangible pathway to resolve issues beyond status-reporting progress to my supervisor.

4.3.1 Risk Management

Due to the practical nature of the project, several tangible challenges stemmed from the project constraints that required immediate mitigation.

- **Risk:** Preconfigured Neutral density filter was too strong
- **Mitigation:** Found a weaker filter, used as the scattering diffuser inherently weakened laser power propagation, which was found to cause attenuation that wasn't optimal for quality image capture, a workaround was seen to be suboptimal, as extreme exposure or gain increases wouldn't result in any meaningful increases in camera quality suitable for sensitive Machine Learning inference

- **Risk:** Spurious light and background lighting discrepancies in data acquisition process

- **Mitigation:** Data acquisition was not an instant process, meaning that data could be sensitive to light changes and the monitor of my setup could also cause unintended light projections, creating a somewhat unfair environment for the Machine Learning models to learn inference. Mitigation was realised in the baffle ring, section 2.1.1b, that allowed a matte black tunnel to protect against unwanted light entering the lenses or the camera.

- **Risk:** MATLAB configuration issues with ZWO camera integration
- **Mitigation:** Mitigated by initially allowing camera native software to confirm USB connectivity, followed by driver installation for ZWO astronomical camera, allowing for MATLAB's Image acquisition Toolbox connectivity, and mitigation of latency, affecting camera-MATLAB synchronisation being handled by adding a buffer frame interval between cameras, trading instant camera capture for synchronisation.

- **Risk:** Computational constraints deemed to render training process intractable
- **Mitigation:** Available hardware with no dedicated GPU would result a required parameter count and model depth to be kept to a minimum, with models made intentionally shallow and memory friendly to fit the specifications of my model.

Risks and mitigation strategies coincide with the monthly progresses in the review proforma, included in 6.1 Proforma, which document the trajectory of challenges faced monthly with initial hardware setup encapsulated with the final specification validation process. Mitigations provided a crucial improvement to the project basis, with the baffle ring employed greatly mitigating the issue faced of light differences due to different periods of the day interfering with the intensity of the captured data, and the Neural density filter allowing for a more streamlined preprocessing workflow, considerably reducing time spent in that phase, helping me to go closer to the fixed timeline envisioned. Similarly, in a future hardware-dependant project, the committal to acquiring datasets or interacting with the experimental environment would be a standard practice procedure in efficient professional scheduling.

Progress Review 1 occurred to change the methodology specified in my drafted proposal given assumptions that were incorrect. Initially behind, there was a revised inclusion of a fixed diffuser, alongside the experimental design to commit to projecting a single, well characterised scattering condition in the project tenure. Attempting to have a setup that had a configurable amount of different specification diffusers wasn't feasible as I realised with only 1 diffuser available to my setup, which necessitated a pivot to analysis revolving that single scattering diffuser and its characteristics. Progress Review 2 occurred to change the hardware integration identification, with a different material being replaced in the experimental setup, along with accepting an exchange of a consistently calibrated image with a longer acquisition period, which was exchanged for the assurance that latency issues wouldn't be present in my dataset, as 10,000 samples of data would be infeasible to manually search through. Progress review 3 introduces the configuration of the optical integration with MATLAB, ultimately resulting in the optimal src.Exposure and src.Gain settings that would be used for the entire dataset.

Progress Review 4 and 5 transitions to a near completed project state, with the dataset being partitioned and expanded upon, alongside the model training aggregated to record metrics that would facilitate in comparative analysis of function, fulfilling **D5**.

4.3.2 Skills Development

The project, physical and computational in nature, allowed for several competencies that would revolve around informing future engineering projects and professional practice. The initial and most prevalent skill would revolve in engineering code to assess Deep Learning objectives, whilst managing integration with peripheral devices. The realised itself in the camera-MATLAB integration debugging process that allowed for the skills that involve working with previously unseen hardware and software that doesn't initially work as intended. For future projects, I would find it easier to adapt optimal hardware configurations and account

for suboptimal device-to-hardware integration and acknowledge when coding needs to be adjusted to account for employed hardware. Driver installation and limitations that stem from non-optimal setup of different architectures that aren't fully native allowed for the real-world problems seen in engineering settings to emerge, directly transferrable issue solving that would not be uncommon in an industrial setting where integration problems that are similar in nature arise.

In a personal capacity, the project developed the fundamental skill of self-management of a self-directed project over a 7-month period, with mechanisms and discipline to attend compulsory weekly meetings, while prepared progress advanced my workflow that would replicate itself in any long duration engineering project. However, the most significant area of improvement arises from the hardware recognition and diagnosing task that I faced, which took a considerable amount of time to rectify, as it delayed weeks from my workflow and timeline. Testing the full ranges of exposure and identification of better practices to solve capture related issues were problems that took longer to solve than what was expected. In an industrial context, where the project timelines are less fluid and don't allow for deviations from a set path unless stated, due to cost of rework, upfront investment in hardware and validation that it works would be practice that is standard, with this project demonstrating how to approach future tasks that are hardware-dependent.

5) References

[1] J. W. . Goodman, *Speckle phenomena in optics : theory and applications*. SPIE Press, 2020.

[2] P. G. Vaz, A. Humeau-Heurtier, E. Figueiras, C. Correia, and J. Cardoso, 'Laser Speckle Imaging to Monitor Microvascular Blood Flow: A Review', 2016, *Institute of Electrical and Electronics Engineers*. doi: 10.1109/RBME.2016.2532598.

[3] S. Gigan *et al.*, 'Roadmap on wavefront shaping and deep imaging in complex media', Oct. 01, 2022, *Institute of Physics*. doi: 10.1088/2515-7647/ac76f9.

[4] M. Bashkansky and J. Reintjes, 'Statistics and reduction of speckle in optical coherence tomography', 2000. doi: 10.1364/OA_License_v1#VOR.

[5] K. D. Ridley, G. N. Pearson, and M. Harris, 'Improved speckle statistics in coherent differential absorption lidar with in-fiber wavelength multiplexing'. doi: 10.1364/OA_License_v1#VOR.

[6] A. P. Mosk, A. Lagendijk, G. Lerosey, and M. Fink, 'Controlling waves in space and time for imaging and focusing in complex media', May 2012. doi: 10.1038/nphoton.2012.88.

[7] I. Isewon, E. Alagbe, and J. Oyelade, 'Optimizing machine learning performance for medical imaging analyses in low-resource environments: The prospects of CNN-based Feature Extractors', *F1000Res.*, vol. 14, p. 100, Jan. 2025, doi: 10.12688/f1000research.156122.1.

[8] A. Kohan, E. A. Rietman, and H. T. Siegelmann, 'Signal Propagation: The Framework for Learning and Inference in a Forward Pass', *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 6, pp. 8585–8596, Jun. 2024, doi: 10.1109/TNNLS.2022.3230914.

[9] I. M. Vellekoop and A. P. Mosk, 'Focusing coherent light through opaque strongly scattering media', 2007. doi: 10.1364/OA_License_v1#VOR.

[10] S. Li *et al.*, 'Compressively sampling the optical transmission matrix of a multimode fibre', *Light Sci. Appl.*, vol. 10, no. 1, Dec. 2021, doi: 10.1038/s41377-021-00514-9.

[11] S. M. Popoff, G. Lerosey, R. Carminati, M. Fink, A. C. Boccara, and S. Gigan, 'Measuring the transmission matrix in optics: An approach to the study and control of light propagation in disordered media', *Phys. Rev. Lett.*, vol. 104, no. 10, Mar. 2010, doi: 10.1103/PhysRevLett.104.100601.

[12] A. Turpin, I. Vishniakou, and J. d. Seelig, 'Light scattering control in transmission and reflection with neural networks', *Opt. Express*, vol. 26, no. 23, p. 30911, Nov. 2018, doi: 10.1364/oe.26.030911.

[13] 'Create MLP Network for Neural State-Space Object', 2026. [Online]. Available: https://www.mathworks.com/help/ident/ref/idneuralstatespace.createmlpnetwork.html

[14] M. G. Pacheco, 'EEEE3001 Final Year Individual Project Thesis Wavefront Shaping of Light using Digital Micromirror Device: Development of Hardware and Software'.

[15] I. Freund, 'Joseph W. Goodman: Speckle Phenomena in Optics: Theory and Applications', *J. Stat. Phys.*, vol. 130, no. 2, pp. 413–414, Nov. 2007, doi: 10.1007/s10955-007-9440-8.

[16] Texas Instruments, 'DLP4710 0.47 1080p DMD', Datasheet DLPS056C, rev. C, Texas Instruments Incorporated, Dallas, TX, USA, 2023. [Online]. Available: https://www.ti.com/lit/ds/symlink/dlp4710.pdf

[17] 'Thorlabs', 2026. [Online]. Available: https://www.thorlabs.com/shack-hartmann-wavefront-sensors

[18] Z. W. O. ASI, 'ZWO ASI 120MM Mini USB 2.0 Mono Camera', May 2026. [Online]. Available: https://www.firstlightoptics.com/zwo-cameras/zwo-asi-120mm-mini-usb-20-mono-camera.html

[19] M. A. Gauvin and E. R. Freniere, 'Reducing stray light in Opto-Mechanical Systems'.

[20] S. Hochreiter, 'THE VANISHING GRADIENT PROBLEM DURING LEARNING RECURRENT NEURAL NETS AND PROBLEM SOLUTIONS'.

[21] V. Berenz, 'camera-zwo-asi', GitHub repository, Max Planck Institute for Intelligent Systems, 2022. [Online]. Available: https://github.com/MPI-IS/camera_zwo_asi

[22] 'PSNR - Compute peak signal-to-noise ratio (PSNR) between images - Simulink', 2026. [Online]. Available: https://mathworks.com/help/vision/ref/psnr.html

[23] 'What Is a Neural Network?' [Online]. Available: https://uk.mathworks.com/discovery/neural-network.html

[24] A. Kamath, J. Willmann, N. Andratschke, and M. Reyes, 'The impact of U-Net architecture choices and skip connections on the robustness of segmentation across texture variations', *Comput. Biol. Med.*, vol. 197, Oct. 2025, doi: 10.1016/j.compbiomed.2025.111056.

[25] L. Li, D. Tang, X. Chu, X. Yang, and F. Yu, 'ReSeg-UNet: A Reconstruction-Guided Optimization Framework for Enhanced Medical Image Segmentation'. [Online]. Available: https://github.com/Li-gzhu/ReSeg-UNet.git.

[26] H. Zhong and T. Zhang, 'A Theoretical Analysis of Optimistic Proximal Policy Optimization in Linear Markov Decision Processes', Jun. 2023, [Online]. Available: http://arxiv.org/abs/2305.08841

[27] 'Proximal Policy Optimization (PPO) Agents - MATLAB & Simulink - MathWorks United Kingdom'. [Online]. Available: https://uk.mathworks.com/help/reinforcement-learning/ug/proximal-policy-optimization-agents.html

[28] J. Zhu, 'EEEE3001 Final Year Individual Project Thesis Intelligent Control for Laser Projection System Using Heuristic Optimization Algorithms'.

[29] 'Windows Video Hardware - MATLAB & Simulink', 2026. [Online]. Available: https://mathworks.com/help/imaq/windows-video-hardware.html

[30] Z. Zhang, X. Wang, G. Zheng, and J. Zhong, 'Hadamard single-pixel imaging versus Fourier single-pixel imaging', *Opt. Express*, vol. 25, no. 16, p. 19619, Aug. 2017, doi: 10.1364/oe.25.019619.

[31] G. Zhang, 'Massive Energy for Massive GPU Empowering AI | Towards Data Science', May 2024. [Online]. Available: https://towardsdatascience.com/massive-energy-for-massive-gpu-empowering-ai-dff59ae1da44/

6) Appendix

6.1 Proforma

a) Progress Review 1

Summarised Planned State of Project: *[This section should summarise where you expected to be with the project at this review stage according to your Time Plan.]* • Initially behind in image acquisition and preprocessing • At this review stage, the plan indicated the model prototyping and fitting to be done at this stage, finding weights and variables that work with the project • Expected to be evaluating performance of models and to determine the feasibility of adjustments that could be taken to fit stretch goals. (**D4/5**) • The original plan assumed a different methodology that has since changed	**Actual Progress Since Last Review** *[Summarise the actual progress you have made on the project since previous Progress Review (or start of project).* *You should consider relevant deliverables, milestones, and Gannt chart tasks.]* • Close to completion on **D1** • Model parameters fully realised for application phase

Next Steps and Supervisor Feedback

[In this section you should consider the differences between where you planned to be and where you are.

- *How will this affect the project?*
- *What are you going to do to get back on track?*
- *Are there on-going problems that will have a knock-on effect to future tasks?*
- *Is there additional support or resource that is required?*

You should include feedback from your supervisor as discussed in the meeting and their approval of changes.]

- To get back on track I will simply follow my new realised methodology, as I had planned previously that the time taken to train models would be far longer than I hope to spend.
- Supervisor Feedback: Hardware changes obscuring light, use of a singular diffuser to save time/ cut redundancy.

b) Progress Review 2

Summarised Planned State of Project: *[This section should summarise where you expected to be with the project at this review stage according to your Time Plan.]* • Stage of **D1** should have been completed, but drastic changes has affected my methodology workflow	**Actual Progress Since Last Review** *[Summarise the actual progress you have made on the project since previous Progress Review (or start of project).* *You should consider relevant deliverables, milestones, and Gannt chart tasks.]* • Hardware replication of Gabriel Pacheco's optical bench was completed • DMD and ZWO cameras integrated and test captures obtained. • Neutral Density Filter caused excessive attenuation, changed to less strong material

Next Steps and Supervisor Feedback

[In this section you should consider the differences between where you planned to be and where you are.

- *How will this affect the project?*
- *What are you going to do to get back on track?*
- *Are there on-going problems that will have a knock-on effect to future tasks?*
- *Is there additional support or resource that is required?*

You should include feedback from your supervisor as discussed in the meeting and their approval of changes.]

- Continue Hardware calibration and familiarise with camera control system.
- Switch to still captures with settle frames is a pragmatic choice given the hardware limitations.

c) Progress Review 3

Summarised Planned State of Project: *[This section should summarise where you expected to be with the project at this review stage according to your Time Plan.]* • **D1** achieved with goal of moving to **D2** imminently	**Actual Progress Since Last Review** *[Summarise the actual progress you have made on the project since previous Progress Review (or start of project).* *You should consider relevant deliverables, milestones, and Gannt chart tasks.]* • Scattering diffuser introduced that was not present in prior project implementation • Need to now calculate its strength via speckle contrast calculation • Need to perform test to assess the best exposure and gain with interplay.

Next Steps and Supervisor Feedback
[In this section you should consider the differences between where you planned to be and where you are.
- *How will this affect the project?*
- *What are you going to do to get back on track?*
- *Are there on-going problems that will have a knock-on effect to future tasks?*
- *Is there additional support or resource that is required?*

You should include feedback from your supervisor as discussed in the meeting and their approval of changes.]

- Adapt preexisting MATLAB examples and access proper functions src.exposure and src.gain for the camera function.

d) Progress Review 4

Summarised Planned State of Project: *[This section should summarise where you expected to be with the project at this review stage according to your Time Plan.]* • Subsystem test worked and dataset is acquired, after break **MLP** and **PPO** models planned for training • Dataset goals of **D6,D7** folded into dataset, seen as a goal that I want to do • prepare dataset for preprocessing and training runs	**Actual Progress Since Last Review** *[Summarise the actual progress you have made on the project since previous Progress Review (or start of project).* *You should consider relevant deliverables, milestones, and Gannt chart tasks.]* • 10000-pair dataset achieved with various subclasses • testing infrastructure built and data readily preprocessed after finding optimal camera settings

Next Steps and Supervisor Feedback
[In this section you should consider the differences between where you planned to be and where you are.
- *How will this affect the project?*
- *What are you going to do to get back on track?*
- *Are there on-going problems that will have a knock-on effect to future tasks?*
- *Is there additional support or resource that is required?*

You should include feedback from your supervisor as discussed in the meeting and their approval of changes.]

- Implement from here, on track for everything so far, need to proceed with completion.

e) Progress Review 5

Summarised Planned State of Project: *[This section should summarise where you expected to be with the project at this review stage according to your Time Plan.]* • Planned final system testing for performance values • Would need to compile figures to realise the project	**Actual Progress Since Last Review** *[Summarise the actual progress you have made on the project since previous Progress Review (or start of project).* *You should consider relevant deliverables, milestones, and Gannt chart tasks.]* • Models trained with the data with 1000 data samples held out as a partition, all models trained to baseline and metrics computed from training run

Next Steps and Supervisor Feedback

[In this section you should consider the differences between where you planned to be and where you are.

- *How will this affect the project?*
- *What are you going to do to get back on track?*
- *Are there on-going problems that will have a knock-on effect to future tasks?*
- *Is there additional support or resource that is required?*

You should include feedback from your supervisor as discussed in the meeting and their approval of changes.]

- For each test, against the specification, need to compare and provide a PASS/FAIL and determine the justifications of them

6.2 Code Snippets

Code Snippet 1: Camera Exposure Calibration

```
% Test a range of exposure levels to determine optimal capture settings
src = getselectedsource(vid);
src.Gain = 10.3;          % Fix analog gain as consistent baseline

exposureLevels = BASE_EXPOSURE + [-1.0, -0.5, 0, +0.5, +1.0];

for k = 1:length(exposureLevels)
   src.Exposure = exposureLevels(k);   % Set logarithmic exposure value
   pause(1);                  % Allow camera to settle
   trigger(vid);
   frame = getdata(vid, 1);          % Capture single frame
   imwrite(frame, fname);            % Save for visual analysis
end
```

Code Snippet 2: Batch Image Centre-Cropping

```
% Centre-crop: 800x640 → 640x640 by removing 80px from left and right
TRIM_X = (SRC_W - CROP) / 2;   % 80 px removed from each horizontal side
TRIM_Y = (SRC_H - CROP) / 2;   % 0 px (height already 640)

X1 = TRIM_X + 1;
X2 = TRIM_X + CROP;
Y1 = TRIM_Y + 1;
Y2 = TRIM_Y + CROP;

for k = 1:numel(files)
   img     = imread(fullfile(INPUT_DIR, files(k).name));
   cropped = img(Y1:Y2, X1:X2, :);   % Apply centre-crop
   imwrite(cropped, fullfile(OUTPUT_DIR, files(k).name));
end
```

Code Snippet 3: Vertical Flip Correction

```
% Compensate for optical inversion by flipping images top-to-bottom
for k = 1:numel(files)
   fpath = fullfile(INPUT_DIR, files(k).name);
   img   = imread(fpath);
   imwrite(flipud(img), fpath);    % Apply vertical flip and overwrite
end
```

Code Snippet 4: Frame Alignment - Index Shifting

```
% Shift input filenames left by one to correct trigger latency offset
delete(fullfile(folder, inputNames{sortOrder(1)}));    % Remove null frame

for i = 1:numel(inputNames)
   movefile(fullfile(folder, inputNames{i}), ...
            fullfile(folder, sprintf('TEMP_input_%05d.png', i)));
end

for i = 1:numel(inputNames)
   movefile(fullfile(folder, sprintf('TEMP_input_%05d.png', i)), ...
            fullfile(folder, sprintf('input_%05d.png', i)));
end
```

Code Snippet 5: MLP Forward Model Architecture

```
% Forward model: Speckle (64x64) → DMD (32x32 binary)
layers_forward = [
   featureInputLayer(4096, 'Normalization', 'none', 'Name', 'speckle_in')
   fullyConnectedLayer(2048, 'Name', 'fc1')
   batchNormalizationLayer('Name', 'bn1')
   reluLayer('Name', 'relu1')
   dropoutLayer(0.3, 'Name', 'drop1')
   fullyConnectedLayer(1024, 'Name', 'fc2')
   batchNormalizationLayer('Name', 'bn2')
   reluLayer('Name', 'relu2')
   dropoutLayer(0.2, 'Name', 'drop2')
   fullyConnectedLayer(1024, 'Name', 'fc_out')
   sigmoidLayer('Name', 'sigmoid_out')
```

```
    regressionLayer('Name', 'regression_out')
];
```

Code Snippet 6: PPO Model Architecture

```
% PPO model: PCA-compressed speckle (256 dims) → DMD (32x32 binary)
% Actor network with Gaussian policy for continuous action space
actor = rlContinuousGaussianActor(actor_net, obs_info, act_info, ...
    ActionMeanOutputNames             = "a_mean_out", ...
    ActionStandardDeviationOutputNames = "a_std_out", ...
    ObservationInputNames             = "obs_in");

% Critic network estimating expected reward for policy updates
critic = rlValueFunction(critic_net, obs_info, ...
    ObservationInputNames = "c_obs_in");
```

Code Snippet 7: MLP Sub-System Testing

```
% Validate trained MLP against baseline: check reconstruction and accuracy
mse_model    = mean((Y_true(:) - double(Y_pred(:))).^2);
mse_baseline = mean(Y_true(:).^2);                % All-zeros baseline

bin_acc = mean(Y_pred_bin(:) == Y_true(:));       % Binary pixel accuracy

% Pixel-wise correlation between predicted and true DMD pattern
for s = 1:size(Y_test,1)
    c = corrcoef(Y_pred(s,:)', Y_true(s,:)');
    r_vals(s) = c(1,2);
end
mean_r = mean(r_vals, 'omitnan');
```

Code Snippet 8: Per-Sample Metric Computation

```
% Compute per-sample SSIM, PSNR, Speckle Contrast K, and Jaccard Index
for i = 1:n
    gt   = reshape(Y_true(i,:), out_sz);
    pred = reshape(Y_bin(i,:),  out_sz);

    ssim_v(i) = ssim(pred, gt);
    psnr_v(i) = psnr(pred, gt, 1.0);

    if mean(pred(:)) > 1e-6
        k_v(i) = std(pred(:)) / mean(pred(:));    % Speckle contrast
    end

    inter = sum(pred(:) & gt(:));
    uni   = sum(pred(:) | gt(:));
    jac_v(i) = inter / max(uni, 1);               % Jaccard (IoU)
end
```